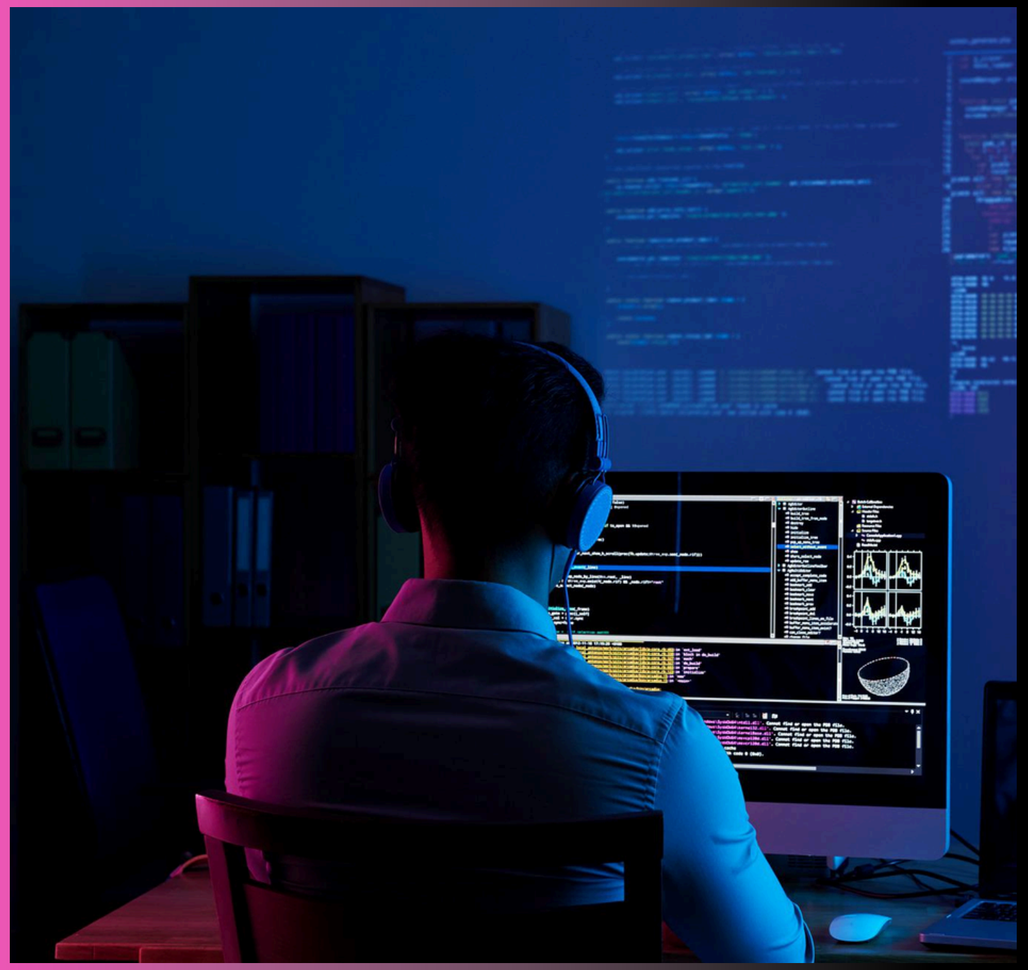
Os originais



Engenharia de Software

Universidade Municipal
de São Caetano do Sul

Integrantes

Alessandro Moreno Ciosani

Guilherme Araújo Almeida

João Victor Corazzari Borges

Lucas Quinto Roli

Vitor Hugo Alvarenga Facciolo

Wendell Pereira de Amorim

Índice

1. Metodologia Ágil

- O que é ?
- Qual a diferença entre um fluxo de trabalho Ágil e um fluxo tradicional?
- Quais são os tipos de fluxos de trabalho ágeis?
- Atores: Product Owner (PO), Scrum Master (SM) e Desenvolvedores
- PMBOK: O que é e suas fases

2. Fluxo de Software

- Linear, Interativo, Evolucionário e Paralelo
- Os principais de Modelos de Processos de Software

3. Elicitação de requisitos

- O que é ?
- Técnicas de elicitação de requisitos.
- Requisitos Funcionais e Não funcionais

4. UML

- O que é ?
- Principais Diagramas UML e Suas Aplicações: Diagrama de caso de uso, Diagrama de Classes, Diagrama de Sequência, Diagrama de Atividades, Diagrama de Componentes e Diagrama de Implementação

Metodologia Ágil

O que é?

A metodologia Ágil otimiza o gerenciamento de projetos ao dividir demandas complexas em etapas menores e gerenciáveis chamadas sprints. Em vez de executar todo o projeto de uma só vez, as equipes trabalham em ciclos curtos que conferem maior rapidez e flexibilidade para corrigir falhas e adaptar o planejamento. Ao final de cada iteração, o feedback coletado dos membros da equipe e partes interessadas direciona as próximas fases, promovendo um fluxo de melhoria contínua focado nos melhores resultados para o negócio.

Qual a diferença entre um fluxo de trabalho Ágil e um fluxo tradicional?

Aspecto	Fluxo Tradicional (Cascata)	Fluxo Ágil
Estrutura	Sequencial e linear	Iterativo e dividido em ciclos (sprints)
Planejamento	Totalmente definido antes do início	Flexível e atualizado continuamente
Execução	Dependente da conclusão da etapa anterior	Entregas contínuas de pequenas partes
Mudanças	Rígidas, exigindo processos formais lentos	Rápidas e integradas ao final de cada ciclo
Feedback	Ocorre somente ao fim de todo o projeto	Coletado frequentemente após cada sprint

Os principais frameworks, metodologias e práticas do universo Ágil

Kanban

Uma abordagem de gestão visual baseada em fluxo contínuo. Utiliza quadros (colunas como "A Fazer", "Fazendo", "Feito") para mapear o processo e foca fortemente em limitar o trabalho em progresso (WIP) para evitar gargalos e garantir que as tarefas sejam finalizadas antes de novas serem iniciadas.

Os principais frameworks, metodologias e práticas do universo Ágil

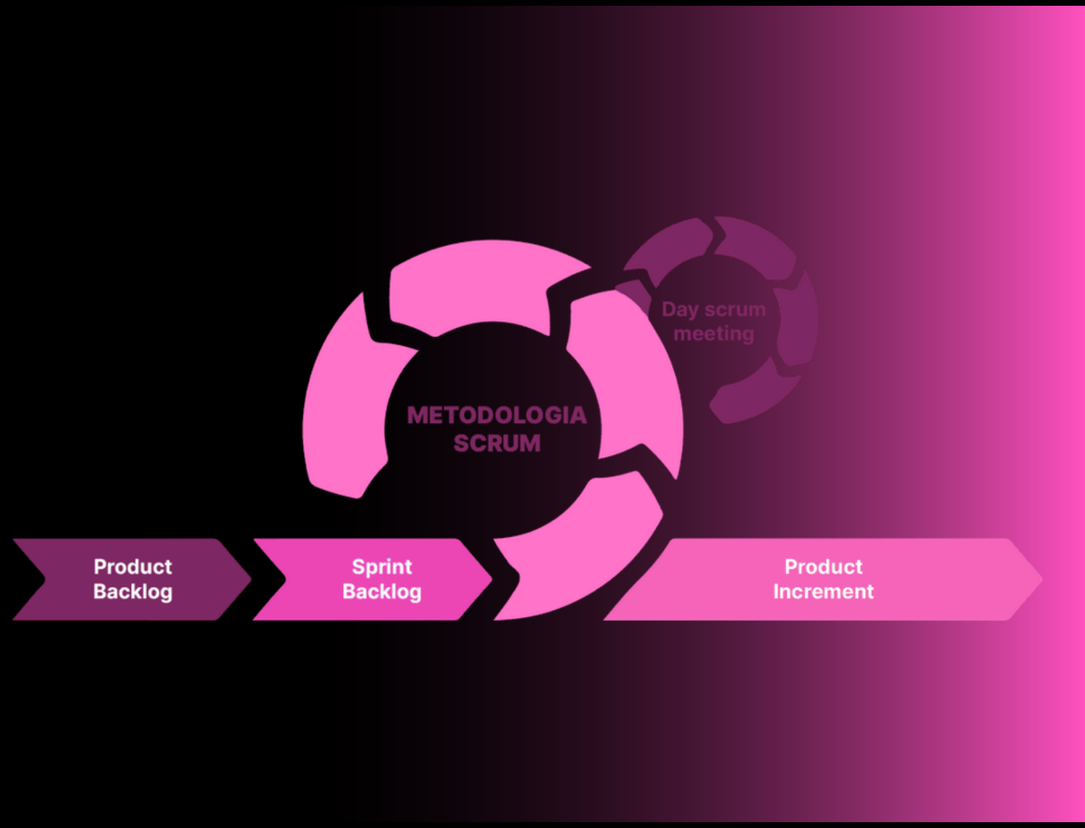
Lean

Originado no sistema de produção da Toyota, o Lean no desenvolvimento de software foca na eliminação de desperdícios (tudo que não agrega valor ao cliente). Seus pilares incluem entregar rápido, adiar decisões irreversíveis até o último momento seguro, focar na qualidade e otimizar o sistema como um todo.

Os principais frameworks, metodologias e práticas do universo Ágil

Scrum

O framework ágil mais popular. Divide o trabalho em ciclos curtos e fixos chamados Sprints (geralmente de 1 a 4 semanas). Define papéis claros (Product Owner, Scrum Master e Desenvolvedores) e ritos específicos (Planejamento, Daily, Revisão e Retrospectiva) para garantir entregas incrementais de valor.



Os principais frameworks, metodologias e práticas do universo Ágil

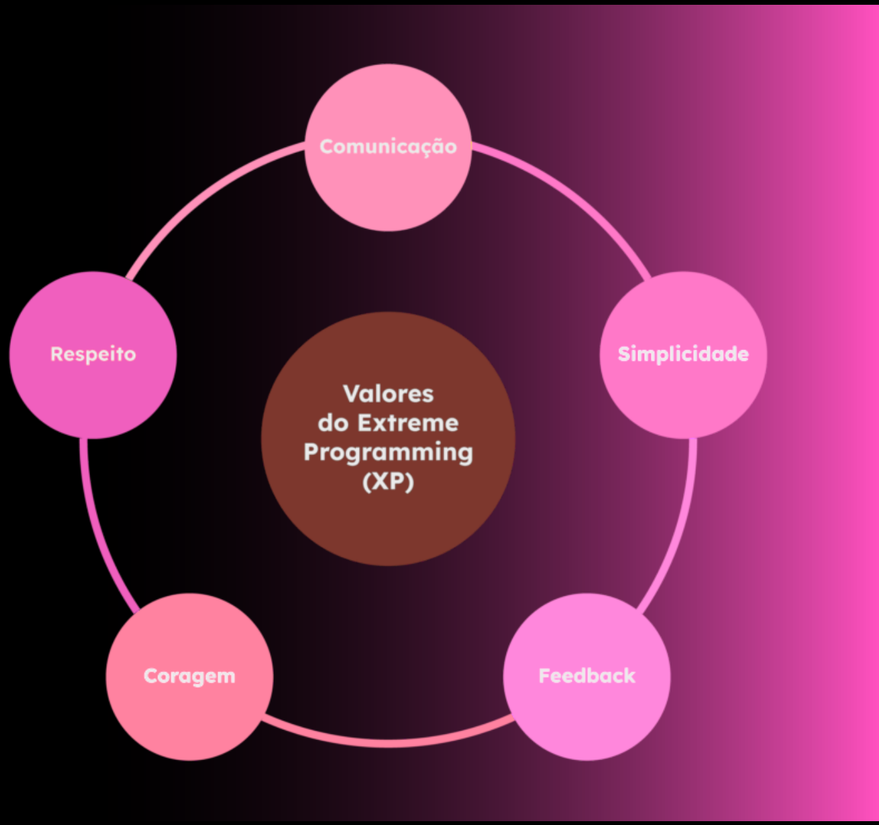
Scrumban

Um híbrido que combina a estrutura do Scrum com a fluidez do Kanban. Geralmente adota as cerimônias e papéis do Scrum, mas remove a rigidez dos Sprints em favor do fluxo contínuo e dos limites de WIP do Kanban. Muito usado por equipes de suporte ou manutenção.

Os principais frameworks, metodologias e práticas do universo Ágil

Extreme Programming (XP)

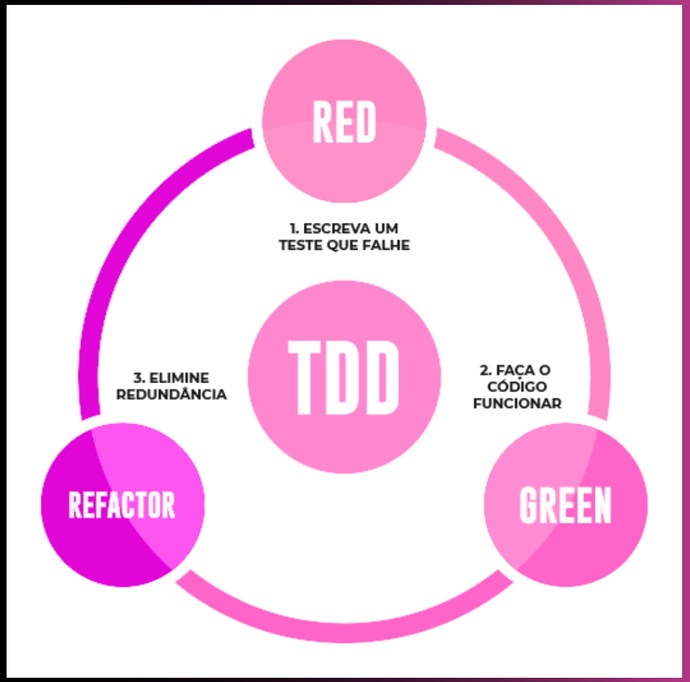
Uma metodologia ágil focada intensamente na excelência técnica e qualidade do código. Promove práticas rigorosas de engenharia, como programação em par (Pair Programming), integração contínua, refatoração constante e entregas muito frequentes para garantir alta adaptabilidade.



Os principais frameworks, metodologias e práticas do universo Ágil

Test Driven Development (TDD)

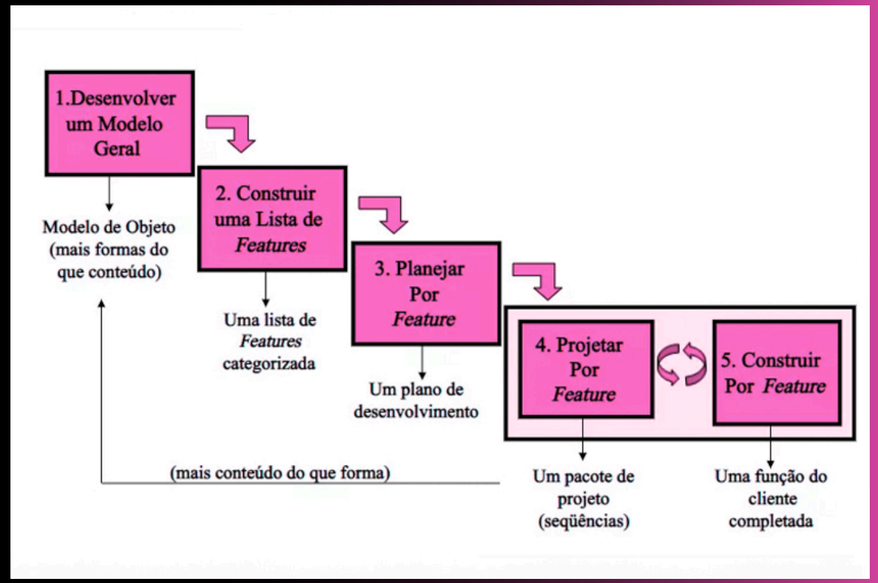
(Desenvolvimento Orientado a Testes) Não é um framework de gestão, mas uma prática técnica. O programador escreve o teste automatizado antes de escrever o código da funcionalidade. O ciclo é: escrever um teste que falha (Red), escrever o código mínimo para o teste passar (Green) e melhorar o código (Refactor).



Os principais frameworks, metodologias e práticas do universo Ágil

Feature Driven Development (FDD)

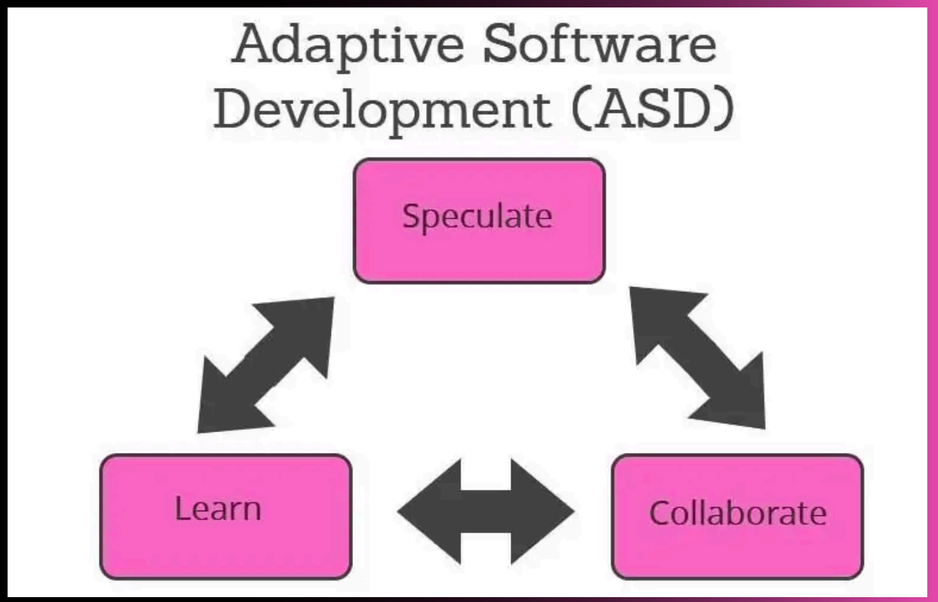
Metodologia orientada por modelos e focada no design e construção por funcionalidades (features). O processo começa com a modelagem do domínio do negócio, seguida pela criação de uma lista de funcionalidades e, então, o planejamento e construção iterativa de cada uma delas.



Os principais frameworks, metodologias e práticas do universo Ágil

Adaptive Software Development (ASD)

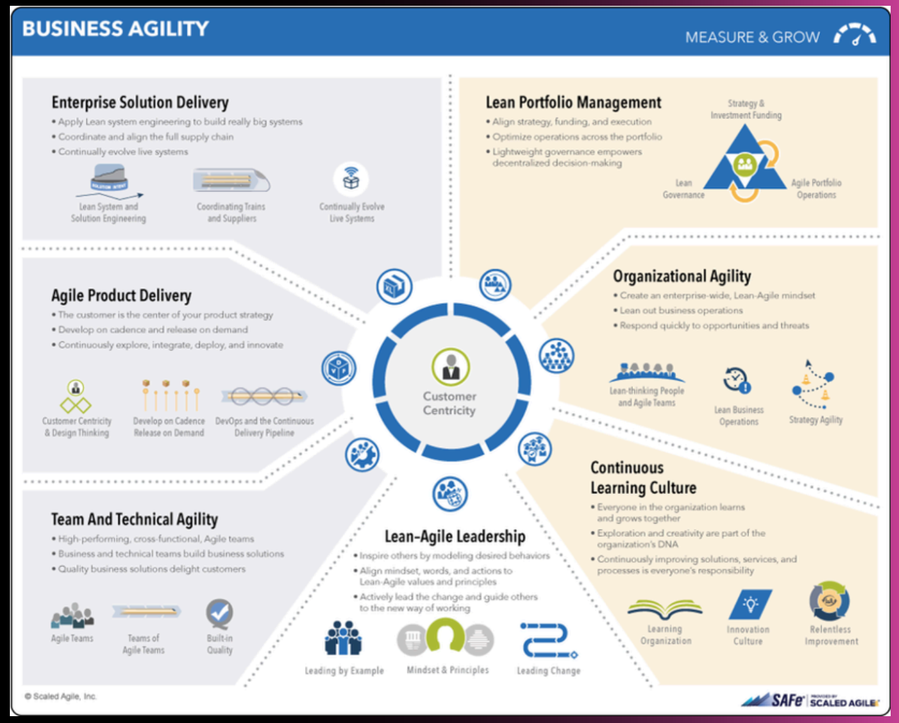
Baseia-se na ideia de que o desenvolvimento é complexo e imprevisível. Substitui o ciclo tradicional de "Planejar-Construir-Testar" pelo ciclo Especular (Speculate), Colaborar (Collaborate) e Aprender (Learn), focando em adaptação constante em vez de otimização rígida.



Os principais frameworks, metodologias e práticas do universo Ágil

Scaled Agile Framework (SAFe)

Um framework robusto criado para escalar o Ágil em grandes corporações. Ele pega práticas do Scrum, Kanban, Lean e XP e fornece uma estrutura para aplicá-las em múltiplos times simultaneamente, alinhando a estratégia de negócios da alta gestão com a execução técnica de centenas de desenvolvedores.



Atores

	Product Owner (PO)	Scrum Master (SM)	Desenvolvedores
O que faz	Define a visão do produto e gerencia o backlog.	Facilita os processos do Scrum.	Constrói o incremento do produto.
Função principal	Maximizar o valor do produto entregue.	Atuar como um líder servidor para o time.	Executar o trabalho técnico com qualidade.
Responsabilidades	Priorizar tarefas, representar o cliente e aceitar ou rejeitar as entrega.	Remover impedimentos, garantir a produtividade e orientar a equipe sobre as práticas ágeis.	Planejar a sprint, criar o produto de forma autogerenciável e realizar testes contínuos.

PMBOK

PMBOK significa Project Management Body of Knowledge (Conjunto de Conhecimentos de Gerenciamento de Projetos na tradução). Trata-se de um guia que descreve práticas, diretrizes e até terminologias padronizadas no setor de gestão de projetos.

Vale destacar que o PMBOK não é uma **metodologia**, mas uma estrutura que organiza conhecimentos e orienta a aplicação de práticas de gerenciamento de projetos, podendo ser adaptado à realidade de cada organização.

Os Seis Princípios de Gerenciamento de Projetos

Princípio	O que enfatiza
Adote uma visão holística.	Incentiva o pensamento sistêmico e a consciência de como as decisões do projeto afetam o ambiente organizacional mais amplo.
Foque no valor	Reforça a ideia de que o sucesso de um projeto é medido por resultados que importam para as partes interessadas, e não apenas pela conclusão de tarefas.
Qualidade de incorporação	Promove a integração proativa da qualidade tanto nos processos quanto nos entregáveis ao longo de todo o ciclo de vida do projeto.
Seja um líder responsável	Combina comportamentos de gestão responsável e liderança para apoiar decisões responsáveis e uma governança ética de projetos.
Construir uma cultura de empoderamento	Enfatiza as considerações ambientais, econômicas e sociais como parte da gestão responsável de projetos.
Construir uma cultura de empoderamento	Promove a colaboração, a confiança, a segurança psicológica e um forte desempenho da equipe.

Questão 1

Uma equipe de desenvolvimento de software adotou o Scrum para gerenciar a construção de um aplicativo financeiro. A metodologia ágil otimiza o gerenciamento de projetos ao dividir demandas complexas em ciclos curtos chamados sprints. Durante a execução do projeto, observou-se que o João era o responsável exclusivo por definir a visão do produto, entender as necessidades dos usuários, gerenciar e priorizar dinamicamente a lista de tarefas do projeto (Product Backlog). Por outro lado, a Maria focava em facilitar os processos ágeis, orientar a equipe, atuar como uma líder servidora e remover ativamente os impedimentos que travavam o progresso do time técnico. A partir das responsabilidades descritas, assinale a alternativa que identifica corretamente os papéis desempenhados por João e Maria, respectivamente, dentro do framework Scrum.

A) João é o Scrum Master (SM); Maria é o Product Owner (PO).
B) João é o Líder Técnico de Desenvolvedores; Maria é a Gerente de Projetos (PMBOK).
C) João é o Product Owner (PO); Maria é a Scrum Master (SM).
D) João é o Gerente de Portfólio; Maria é a Desenvolvedora sênior.
E) João e Maria compartilham o papel de Product Owner (PO).

Questão 1

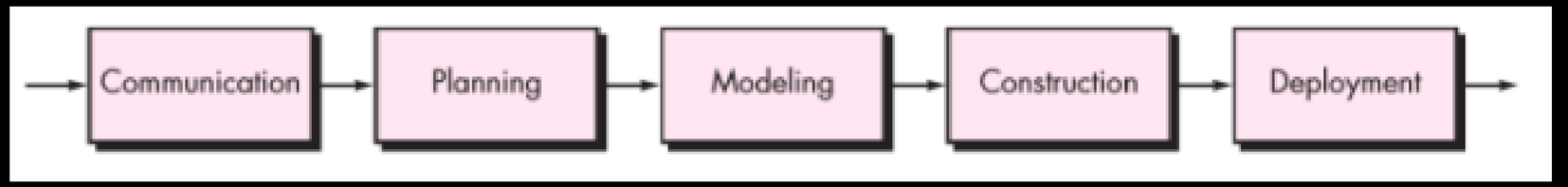
Gabarito: C) João é o Product Owner (PO); Maria é a Scrum Master (SM).

Justificativa: No Scrum, o Product Owner (PO) é responsável por definir a visão do produto, maximizar o valor entregue, e gerenciar/priorizar o Product Backlog (lista de tarefas do projeto), perfil exato das atividades do João. O Scrum Master (SM) atua como líder servidor, facilita os processos, orienta a equipe sobre práticas ágeis e atua para remover impedimentos, papel desempenhado pela Maria. Os Desenvolvedores são responsáveis por construir o incremento do produto técnico.

Fluxo de Software

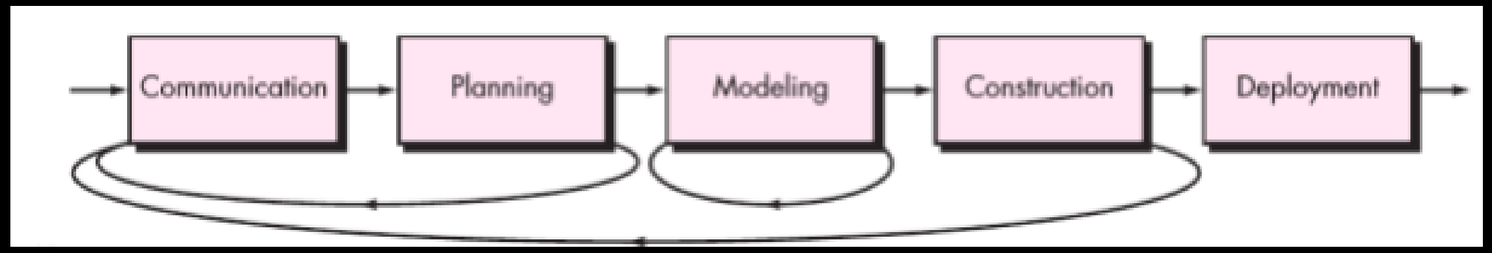
Linear

Trata-se de um modelo rígido e estritamente sequencial, no qual cada etapa do levantamento de requisitos à manutenção pós-lançamento deve ser totalmente concluída antes do início da próxima, sem permissão para retornos ao longo do processo.



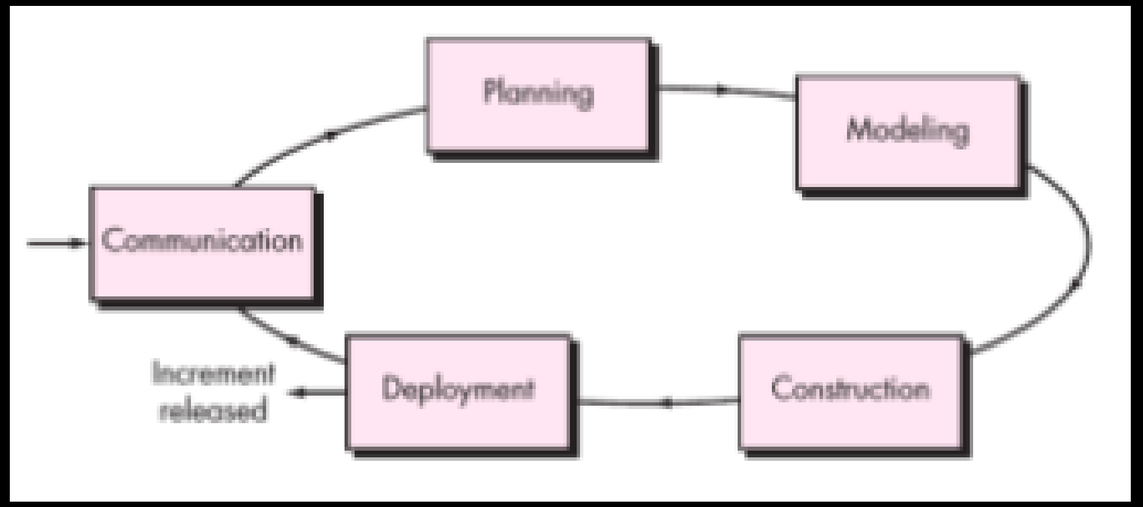
Iterativo

Permite revisar, repetir tarefas e retornar a etapas anteriores sempre que necessário, garantindo flexibilidade para corrigir falhas e incorporar melhorias ao longo do desenvolvimento.



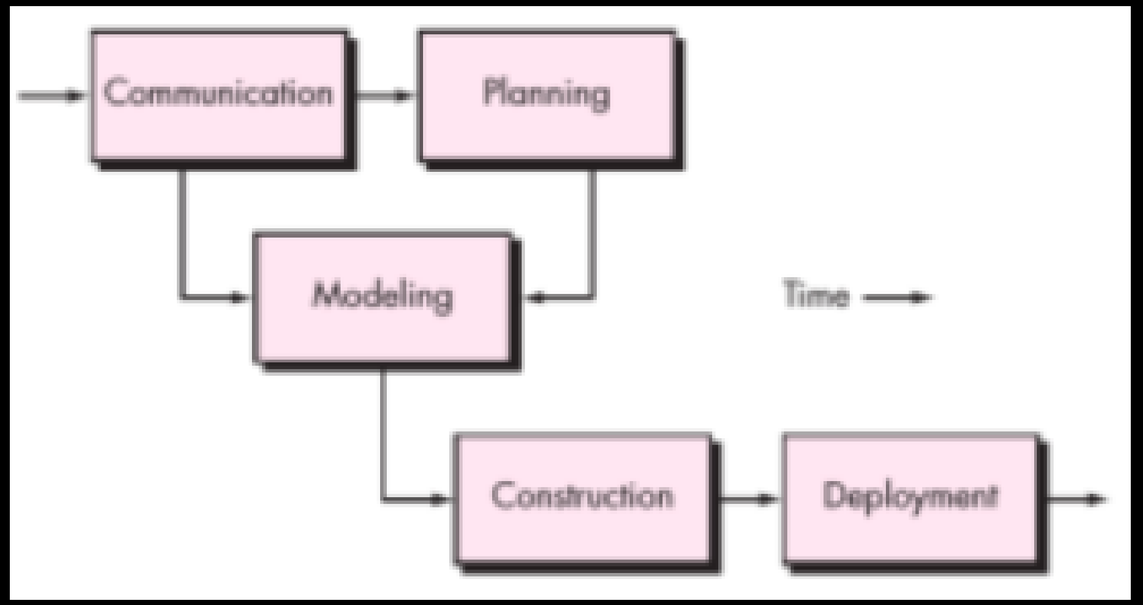
Evolucionário

O modelo evolucionário desenvolve o software em ciclos contínuos de planejamento, criação, testes e refinamento baseado no feedback dos usuários, permitindo que o produto evolua e se adapte às necessidades do público ao longo do tempo.



Paralelo

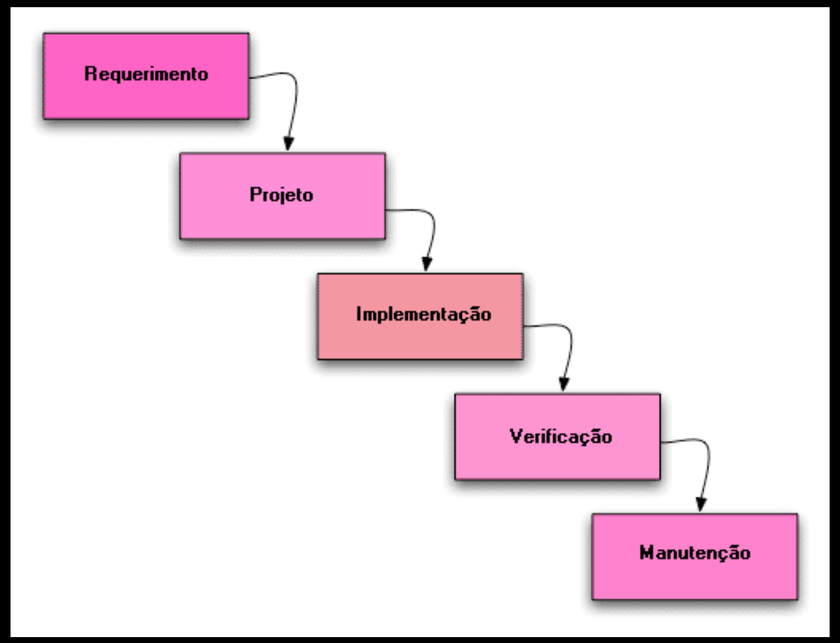
O modelo paralelo executa várias etapas do projeto simultaneamente por meio de equipes distintas, permitindo que tarefas como projetos, codificação e testes avancem ao mesmo tempo para acelerar e otimizar o desenvolvimento.



Os principais de Modelos de Processos de Software

Modelo Cascata

É um processo linear e rígido no qual o desenvolvimento ocorre em fases sequenciais do levantamento de requisitos até a manutenção sem permitir o retorno ou revisão de etapas já concluídas.



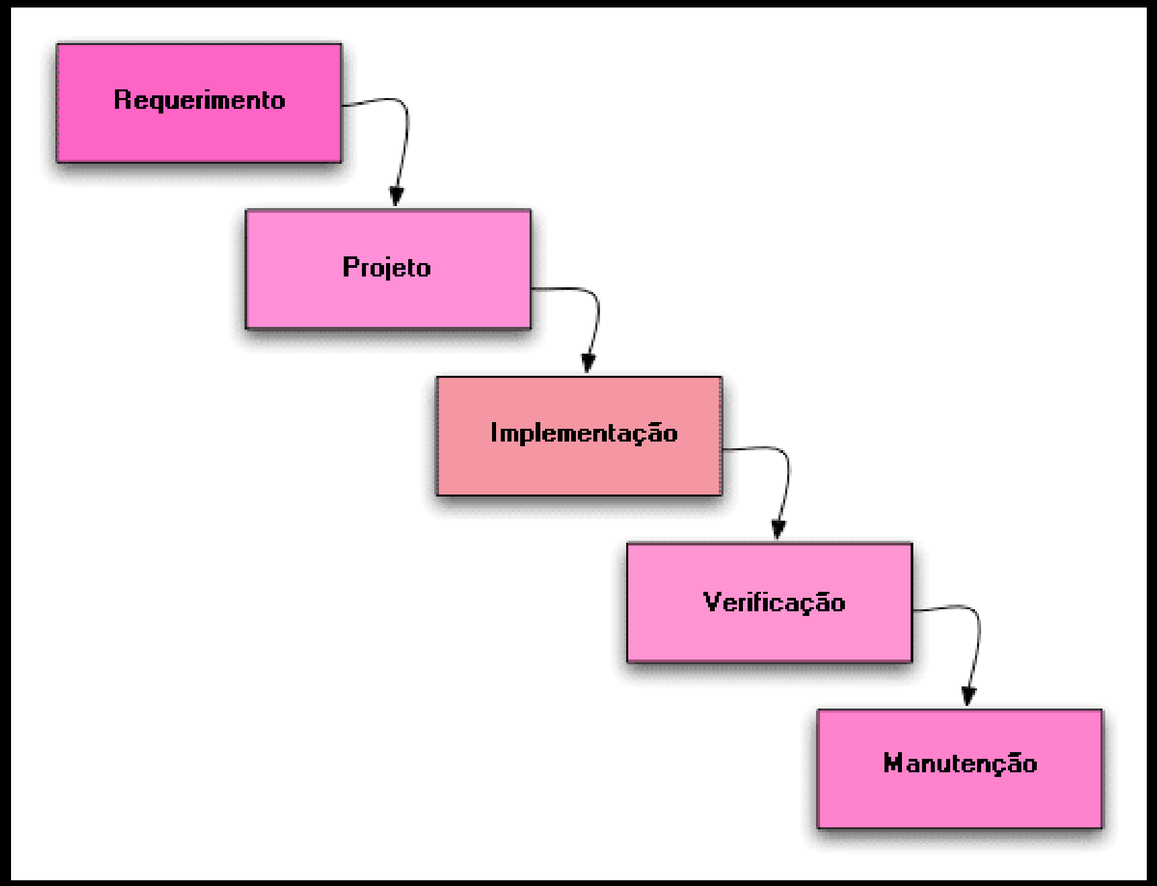
Principais características do Modelo Cascata

- **Estrutura linear e sequencial:** como uma cachoeira, o fluxo segue sempre para frente;
- **Documentação extensa:** cada fase gera documentos detalhados;
- **Controle rígido de escopo:** o que foi definido no início raramente muda;
- **Previsibilidade de custos e prazos:** orçamento e cronograma fixos desde o início;
- **Baixa participação do cliente:** envolvimento principalmente no começo e no fim.

Inicialmente, o modelo waterfall foi proposto como uma maneira de melhorar a previsibilidade dos projetos. Como ele é baseado em planejamento detalhado e documentação completa, oferece maior capacidade preditiva sobre tempo e orçamento para as empresas.

Modelo Cascata – Estágios ou Fases

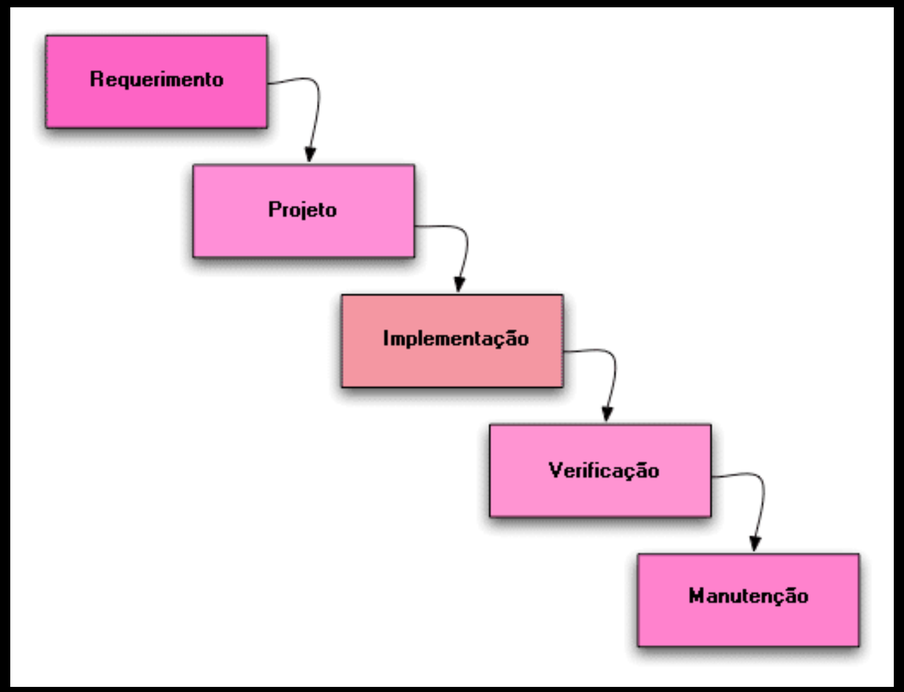
- Definição e Análise de Requisitos
- Projeto do Software
- Implementação e Testes de Unidade
- Integração e Testes de Sistema
- Operação e Manutenção



Desafios

- O principal desafio do modelo cascata é a dificuldade de acomodar mudanças após o processo estar em execução. Em princípio, uma fase tem de ser concluída antes de se mover para a próxima fase.

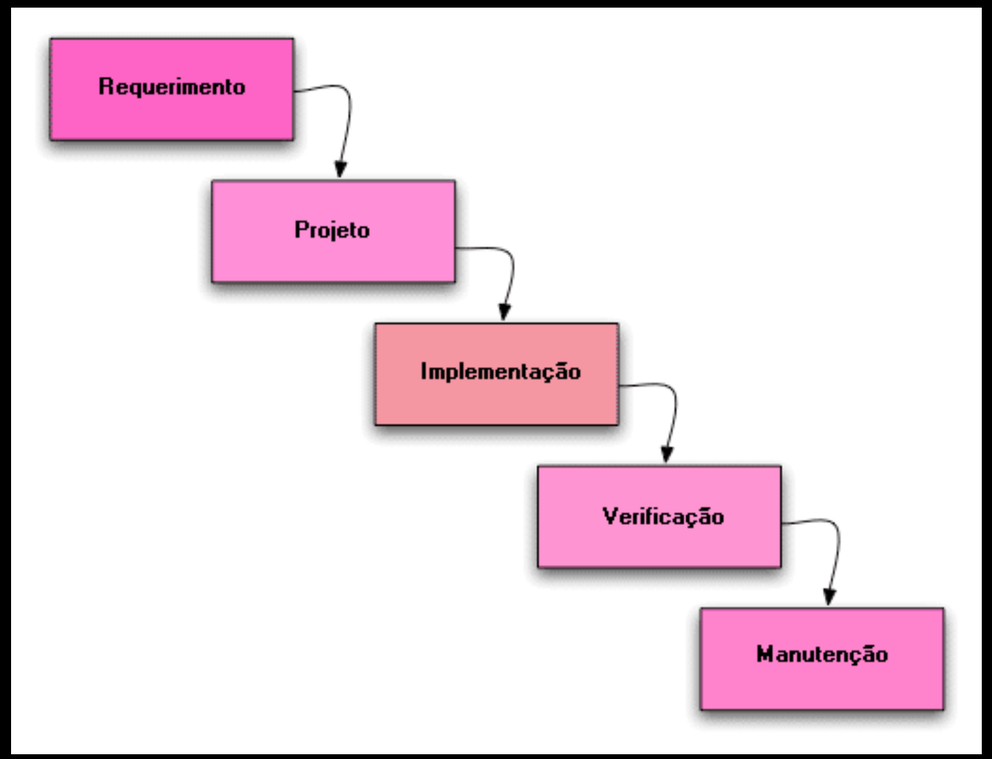
- Maior dificuldade em responder à mudança de requisitos.



Onde o modelo cascata é mais empregado ?

- O modelo é apropriado em sistemas nos quais os requisitos são bem compreendidos e alterações do escopo do sistema estarão limitadas durante a fase de design.

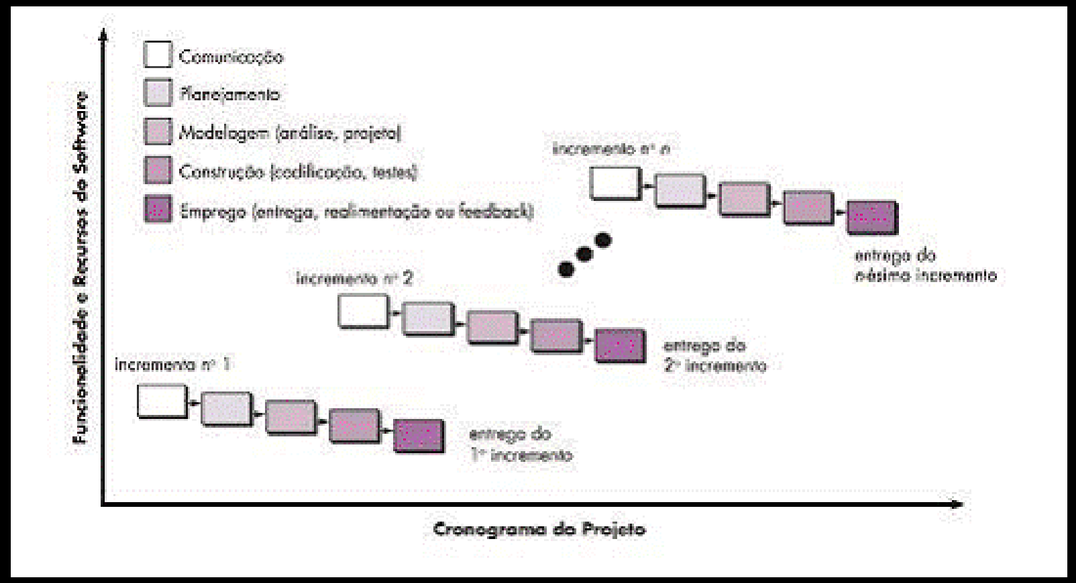
- Poucos sistemas de negócios têm requisitos estáveis.



Desenvolvimento Incremental

Constrói o software em pequenas partes (incrementos) planejadas, codificadas e testadas individualmente, o que acelera as entregas e permite adicionar funcionalidades gradualmente ao produto.

- O modelo incremental combina elementos do fluxo de processos lineares e paralelos.
- Aplicam-se sequências lineares, de forma escalonada, à medida em que o tempo vai avançando.
- Cada sequência linear gera “incrementais” (entregáveis/aprovados/liberados)



Desenvolvimento Evolucionário

- Constrói o software em ciclos progressivos, aprimorando o produto com base no feedback constante dos usuários. Ele abrange dois modelos principais:

- Conforme o desenvolvimento do projeto avança, as necessidades de negócio e de produto podem mudar, tornando inadequado o fluxo linear de atividades.

- Nessas situações, deve-se usar um modelo de processo que tenha sido projetado para desenvolver software que evolua com o tempo.

- Modelos evolucionários são iterativos.

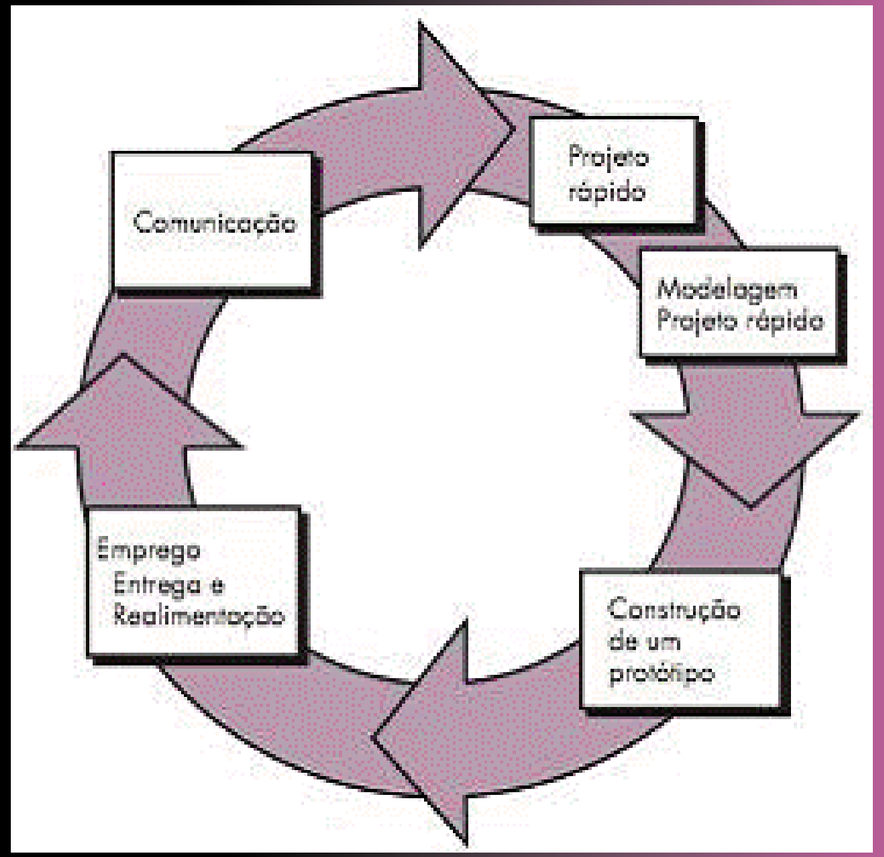
Como desenvolver um software que evolui ao longo do tempo ?

Desenvolvimento Evolucionário

Constrói o software em ciclos progressivos, aprimorando o produto com base no feedback constante dos usuários. Ele abrange dois modelos principais:

Prototipação

Cria versões preliminares e simples para validar o funcionamento com os usuários, permitindo entender melhor os requisitos e aplicar ajustes antes do desenvolvimento definitivo.

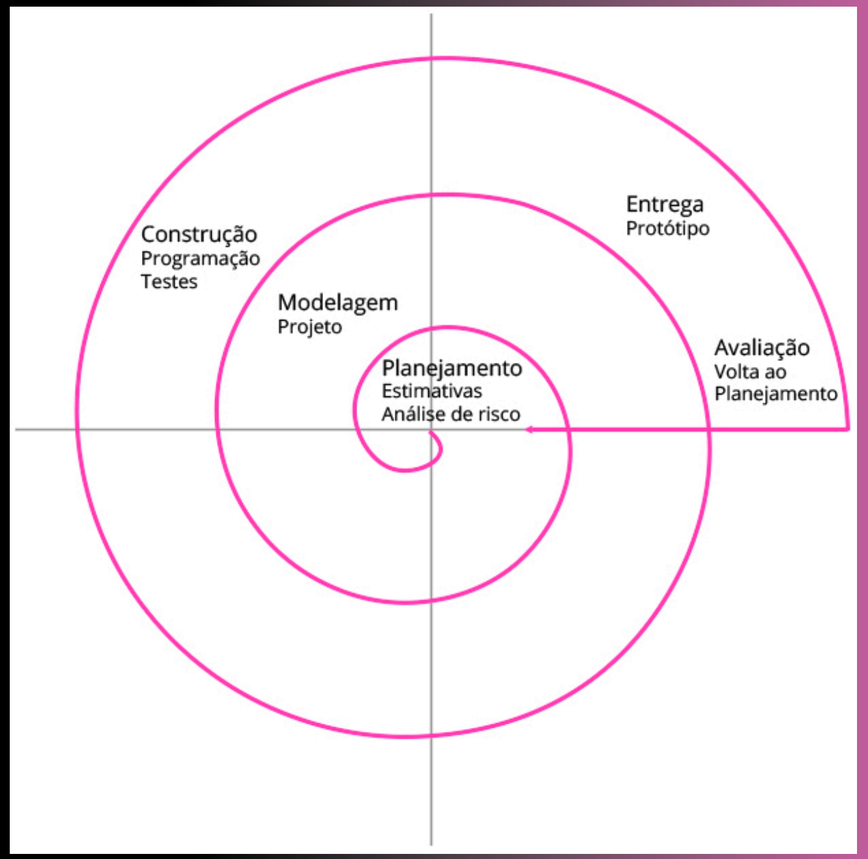


Desenvolvimento Evolucionário

Constrói o software em ciclos progressivos, aprimorando o produto com base no feedback constante dos usuários. Ele abrange dois modelos principais:

Espiral

Desenvolve o sistema em ciclos expansivos que priorizam a análise contínua de riscos e o planejamento a cada etapa, combinando testes e refinamentos sucessivos para garantir maior flexibilidade.



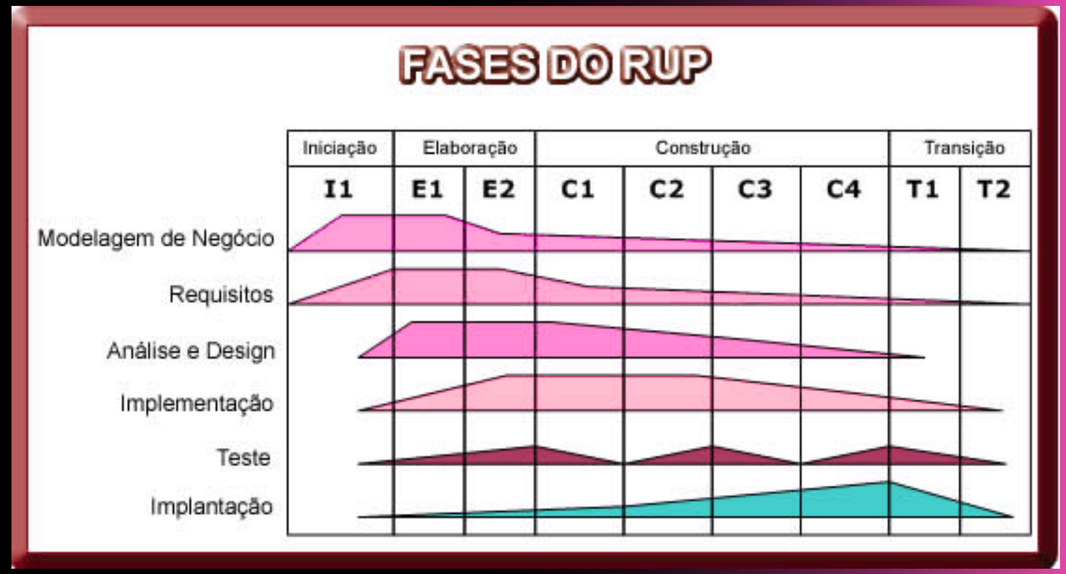
Engenharia de Software Orientada a Reuso

Foca no reaproveitamento de componentes, módulos e bibliotecas já existentes em vez do desenvolvimento do zero, otimizando o tempo e reduzindo o esforço da equipe.

Rational Unified Process (RUP)

É uma metodologia de desenvolvimento de software iterativa e incremental, criada para gerenciar sistemas complexos com flexibilidade. Seu ciclo de vida divide-se em quatro fases principais:

- **Incepção**: Identificam-se as necessidades de negócio para o software.Levantam-se requisitos de negócio.
- **Elaboração:** Desenvolve-se a modelagem do software. Define-se umasolução para o problema do ponto de vista de arquitetura.
- **Construção:** Projeto, programação e testes.
- **Transição:** Libera o sistema para seu ambiente operacional.



Questão 2

Uma startup de tecnologia da área de saúde está desenvolvendo um novo sistema de prontuário eletrônico. Os desenvolvedores adotaram um ciclo de trabalho em que o software é desenvolvido em ciclos contínuos de planejamento, criação, testes e refinamento. Devido à criticidade das informações e à dificuldade inicial dos médicos em expressarem exatamente o que precisavam, a equipe técnica decidiu criar versões preliminares e simples da interface. Essas versões foram disponibilizadas aos médicos para que pudessem validar o funcionamento na prática, permitindo à equipe entender melhor as necessidades reais antes de iniciar a codificação definitiva e complexa do back-end. Com base nos modelos e fluxos de processos de software, avalie as afirmações a seguir sobre a estratégia adotada pela startup:

I. A equipe está utilizando um modelo de fluxo evolucionário, que permite que o produto se adapte às necessidades do público ao longo do tempo com base em feedback constante.
II. A criação de versões preliminares e simples para validação do funcionamento caracteriza a técnica de Prototipação, que ajuda a entender melhor os requisitos antes do desenvolvimento definitivo.
III. O processo descrito exemplifica o modelo Cascata, pois a fase de levantamento de requisitos com os médicos está sendo finalizada completamente antes do início do projeto e da implementação.

É correto o que se afirma em:

A) I, apenas. **B)** I e II, apenas. **C)** II, apenas. **D)** II e III, apenas. **E)** I, II e III.

Questão 2

Gabarito: B) I e II, apenas.

Justificativa: A afirmação I é correta, pois o modelo evolucionário foca em ciclos contínuos de aprimoramento e adaptação com base no feedback dos usuários. A afirmação II é correta, pois a Prototipação consiste exatamente em criar versões preliminares e simples para validar o funcionamento e entender os requisitos antes do desenvolvimento definitivo. A afirmação III é incorreta, pois o modelo Cascata é rígido, sequencial e não permite retorno a etapas anteriores após sua conclusão, o que é o oposto da abordagem iterativa e de validação contínua descrita no cenário.

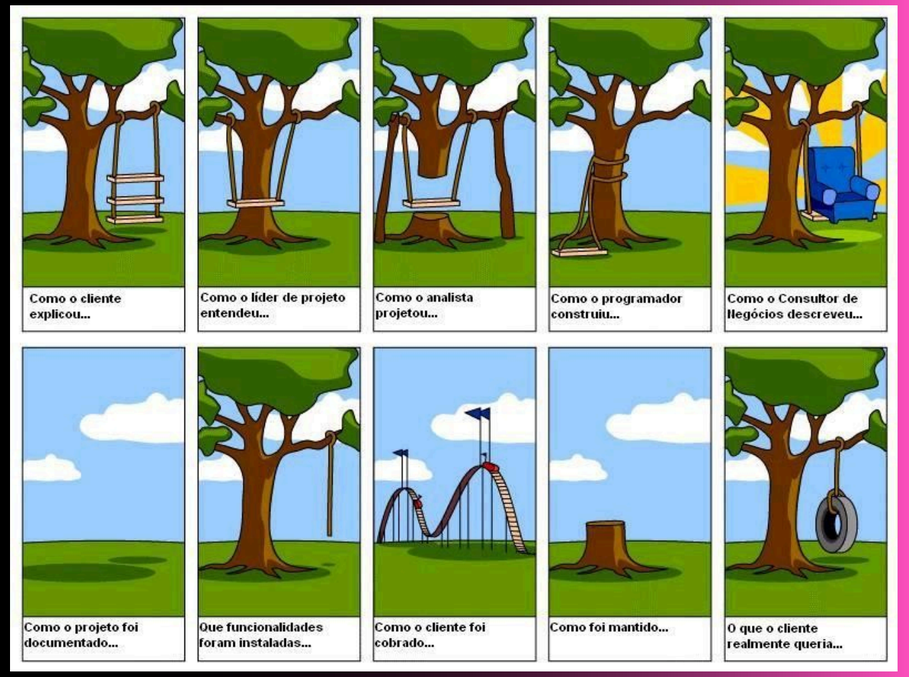
Elicitação de Requisitos

O que é?

Elicitação de requisitos é o nome dado para o conjunto de atividades realizadas para a descoberta dos requisitos de um sistema.

A melhor elicitação é aquela que consegue identificar o maior número possível de requisitos necessários para o desenvolvimento de um sistema.

Assim, não se trata apenas de questionar os clientes e usuários sobre o que eles precisam, mas sim de uma análise da empresa, da área de negócio, dos processos envolvidos e dos problemas enfrentados.



Técnicas comuns de elicitação de requisitos

A elicitação de requisitos utiliza diferentes estratégias para levantar e definir as necessidades de um projeto:

Entrevistas

Diálogos diretos com os stakeholders para levantar desejos e necessidades.

Questionários

Formulários padronizados para coletar informações de um grande volume de pessoas.

Observação

Acompanhamento da rotina dos usuários em seu ambiente de trabalho para identificar necessidades práticas.

Técnicas de Elicitação de requisitos em Grupo

Workshops

Reuniões de trabalho colaborativas para discutir e alinhar requisitos.

Brainstorming

Dinâmica focada na geração livre e sem julgamentos do maior número possível de ideias, que depois são discutidas, selecionadas e documentadas.

JAD (Joint Application Development)

Imersão intensiva conduzida por um facilitador, reunindo usuários, gerentes e desenvolvedores para definir, visualizar e aprovar requisitos com documentação em tempo real.

RAD (Rapid Application Development)

Metodologia focada na entrega rápida de sistemas funcionais por meio do uso combinado de JAD, prototipação e ferramentas de aceleração de desenvolvimento.

Requisitos Funcionais *vs* Requisitos Não Funcionais

Requisitos Funcionais

"O que o sistema faz"

Define as funções explícitas, os recursos operacionais, os fluxos de trabalho e a lógica de negócio que o software deve executar em resposta às ações do usuário ou dados de entrada.

Requisitos Não Funcionais

"Como o sistema se comporta"

Especifica os atributos de qualidade, restrições técnicas, desempenho e ambiente operacional. Garante que as funcionalidades operem de forma ágil, segura e confiável.

Os originais

Referência: ISO/IEC 25010:2023 - “Este modelo pode ser usado para a especificação de requisitos e a avaliação da qualidade de produtos de software ao longo do seu ciclo de vida por diversas partes interessadas, incluindo desenvolvedores, adquirentes, pessoal de garantia e controle de qualidade, além de avaliadores independentes.”

Requisitos Funcionais *vs* Requisitos Não Funcionais

Requisitos Funcionais

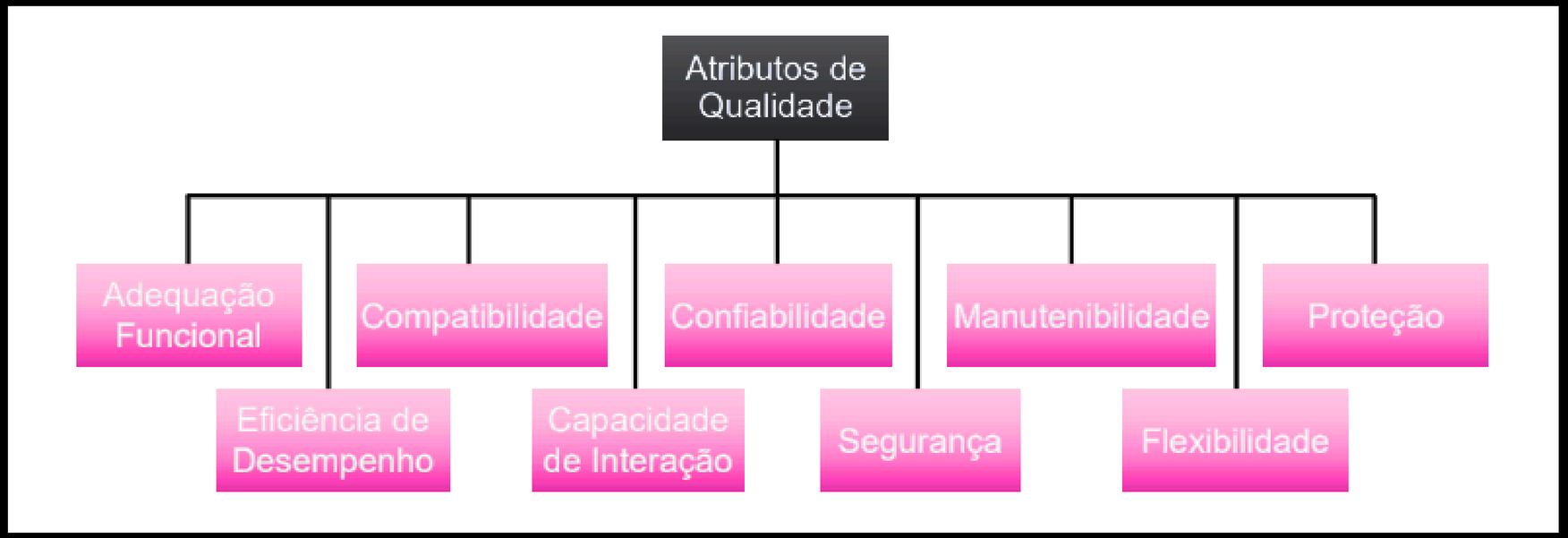
- **Comportamento Direto:** Descrevem com precisão as tarefas e transformações que o sistema executa sobre os dados de entrada para gerar uma saída esperada.
- **Foco no Negócio:** Estão diretamente ligados às necessidades dos usuários e regras operacionais (ex: emissão de relatórios, cadastros e filtros).
- **Exemplos Práticos:** "O sistema deve permitir login via e-mail",

Requisitos Não Funcionais

- **Atributos de Qualidade:** Abrangem desempenho, usabilidade, confiabilidade, segurança, disponibilidade e escalabilidade.
- **Critérios Mensuráveis:** Devem ser especificados quantitativamente para permitir testes e validações precisas no sistema.
- **Exemplos Práticos:** "A busca deve responder em até 2 segundos"

Norma ISO/IEC 25010: 2023 Qualidade do produto SW

Uma estrutura hierárquica define as características e sub- características de Qualidade do Produto SW



As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Adequação funcional

A adequação funcional mede o quão bem o software fornece funções que satisfazem as necessidades declaradas e implícitas do usuário. Em essência, responde a uma pergunta simples: o software executa as funções corretas de forma adequada?

A adequação funcional inclui:

- **Completude funcional:** abrangendo todas as funcionalidades necessárias.
- **Correção funcional:** produzir resultados precisos e esperados.
- **Adequação funcional:** permitir que os usuários alcancem seus objetivos de forma eficiente.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Eficiência de desempenho

A eficiência de desempenho avalia a eficácia com que um software utiliza recursos computacionais, atendendo aos níveis de desempenho exigidos sob condições específicas.

Suas principais subcaracterísticas incluem:

- **Comportamento temporal:** tempo de resposta, latência e taxa de transferência.
- **Utilização de recursos:** uso eficiente de recursos de CPU, memória, armazenamento e rede.
- **Capacidade:** habilidade para lidar com a carga de trabalho e o crescimento esperados.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Compatibilidade

A compatibilidade mede a eficácia com que um software opera em conjunto com outros sistemas, compartilhando dados, infraestrutura e recursos computacionais.

A compatibilidade consiste em:

- **Coexistência:** operar de forma eficiente em conjunto com outros softwares sem conflitos.
- **Interoperabilidade:** troca e utilização de informações entre sistemas e serviços.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Capacidade de interação

Uma das atualizações mais significativas introduzidas na ISO/IEC 25010:2023 é a substituição da usabilidade pela capacidade de interação.

A capacidade de interação avalia a eficácia com que os usuários podem interagir com o software para atingir seus objetivos. Embora abranja princípios tradicionais de usabilidade, essa característica atualizada reconhece que o software moderno deve suportar uma gama mais ampla de usuários, modelos de interação e requisitos de acessibilidade. alcancem seus objetivos de forma eficiente.

A capacidade de interação inclui:

- Reconhecimento de adequação
- Facilidade de aprendizado
- Operacionalidade
- Proteção contra erros do usuário
- Engajamento do usuário
- Inclusão
- Assistência ao usuário autodescritivo

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Confiabilidade

A confiabilidade mede o grau em que um software executa funções específicas de forma consistente, sob condições definidas, ao longo do tempo, sem interrupções ou falhas.

Um software confiável minimiza falhas inesperadas, mantendo a disponibilidade do serviço tanto em condições normais de operação quanto em situações adversas.

A norma ISO/IEC 25010:2023 define quatro subcaracterísticas principais:

- **Impecabilidade**
- **Disponibilidade**
- **Tolerância a falhas**
- **Recuperabilidade**

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Security

A segurança mede o grau em que um software protege informações, sistemas e serviços contra acesso, modificação, divulgação ou interrupção não autorizados, garantindo que apenas usuários e processos autorizados possam executar ações permitidas. É uma característica fundamental da segurança de aplicações, da programação segura e da qualidade do software.

A norma ISO/IEC 25010:2023 define as seguintes subcaracterísticas de segurança:

- **Confidencialidade:** proteger informações contra divulgação não autorizada.
- **Integridade:** prevenção da modificação não autorizada de dados
- **Não repúdio:** garantir que ações e transações não possam ser negadas posteriormente.
- **Responsabilização:** permitir que as ações sejam rastreadas até as entidades responsáveis.
- **Autenticidade:** verificação das identidades de usuários, sistemas e serviços.
- **Resistência:** suportar, responder e recuperar-se de ataques ou ações maliciosas.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Manutenibilidade

A manutenibilidade mede a eficácia com que o software pode ser analisado, modificado, testado e aprimorado ao longo de seu ciclo de vida. Um software altamente manutenível permite que as equipes de engenharia respondam rapidamente às mudanças nos requisitos de negócios, controlando a dívida técnica e preservando a qualidade do software a longo prazo.

A característica de manutenibilidade inclui:

- **Modularidade:** separar a funcionalidade em componentes bem definidos.
- **Reutilização:** aproveitamento de recursos existentes em diferentes sistemas e projetos.
- **Analisabilidade:** diagnóstico eficiente de defeitos, vulnerabilidades e problemas de desempenho.
- **Modificabilidade:** implementar mudanças sem introduzir efeitos colaterais indesejados.
- **Testabilidade:** verificar se as alterações se comportam conforme o esperado.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Flexibilidade

A norma ISO/IEC 25010:2023 substitui a característica anterior de Portabilidade por Flexibilidade, refletindo a crescente necessidade de software que possa evoluir juntamente com as mudanças nas tecnologias, ambientes de implantação e requisitos de negócios. A flexibilidade mede o grau em que o software pode se adaptar a novos ambientes, mudanças nas cargas de trabalho, evolução da infraestrutura e necessidades operacionais futuras.

Suas subcaracterísticas incluem:

- **Adaptabilidade:** capacidade de ajuste a diferentes ambientes e configurações operacionais.
- **Facilidade de instalação:** permitindo implantação e instalação eficientes.
- **Substituibilidade: substituição** de componentes ou sistemas com o mínimo de interrupção.
- **Escalabilidade:** manter o desempenho e a funcionalidade à medida que as cargas de trabalho aumentam.

As nove características de qualidade de produto de software da ISO/IEC 25010:2023

Safety

Uma das adições mais significativas na ISO/IEC 25010:2023 é a Safety , reconhecendo que a qualidade do software vai além da confiabilidade e segurança, incluindo também a proteção contra danos não intencionais. Medidas de segurança referem-se ao grau em que o software evita causar riscos inaceitáveis a pessoas, organizações, propriedades ou ao meio ambiente durante a operação normal e o uso indevido previsível.

A segurança inclui subcaracterísticas como:

- **Restrição operacional:** impedir a operação fora dos limites de segurança.
- **Identificação de riscos: r**econhecer condições perigosas antes que ocorram falhas.
- **Comportamento à prova de falhas:** transição para estados operacionais seguros durante falhas.
- **Aviso de perigo:** informando usuários e operadores sobre condições inseguras.
- **Integração segura:** garantir que as interações com outros sistemas não introduzam riscos inaceitáveis.

Questão 3

Na Engenharia de Requisitos, a distinção clara entre Requisitos Funcionais (RF) e Requisitos Não Funcionais (RNF) é determinante para a sustentabilidade arquitetural. Em muitos projetos ágeis, observa-se a tendência de expressar apenas necessidades funcionais na forma de User Stories (Ex.: "Eu como cliente quero visualizar meu extrato"), negligenciando restrições de desempenho, disponibilidade e escalabilidade.

Analise as asserções a seguir e a relação proposta entre elas:

I. Requisitos Não Funcionais (RNFs) devem ser quantificáveis e testáveis por meio de métricas objetivas (como tempo máximo de resposta em milissegundos ou taxa de disponibilidade em porcentagem), evitando-se formulações ambíguas como "o sistema deve ser rápido e amigável".

PORQUE

II. RNFs orientam diretamente a definição da arquitetura de software e dos atributos de qualidade do sistema, impactando decisões estruturais que raramente podem ser corrigidas apenas com alterações pontuais na camada de código de uma única funcionalidade.

A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.
B) As asserções I e II são proposições verdadeiras, mas a II não é uma justificativa correta da I.
C) A asserção I é uma proposição verdadeira, e a II é uma proposição falsa.
D) A asserção I é uma proposição falsa, e a II é uma proposição verdadeira.
E) As asserções I e II são proposições falsas.

Questão 3

Gabarito: A) As asserções I e II são proposições verdadeiras, e a II é uma justificativa correta da I.

Justificativa: A primeira asserção é verdadeira pois os Requisitos Não Funcionais (RNFs) especificam atributos de qualidade e restrições técnicas que exigem validação objetiva por meio de testes, o que torna indispensável o uso de critérios quantificáveis como tempos limites de resposta ou percentuais de disponibilidade em detrimento de descrições vagas e subjetivas. A segunda asserção também é verdadeira porque os RNFs atuam como os principais direcionadores da arquitetura do sistema, determinando a escolha de infraestruturas, bancos de dados, estratégias de cache e padrões de comunicação, decisões estruturais de alto impacto que não podem ser corrigidas com simples ajustes isolados no código. Dessa forma, a asserção II justifica perfeitamente a asserção I, uma vez que a obrigatoriedade de medir e quantificar os RNFs com precisão existe justamente devido ao peso que essas definições exercem sobre as fundações arquiteturais do software, prevenindo falhas estruturais de difícil reparação ao longo do ciclo de vida do projeto.

UML

O que é UML ?

É uma linguagem de modelagem padronizada que permite visualizar, especificar, construir e documentar sistemas de software. Criada para unificar diferentes métodos de modelagem, ela se tornou um padrão internacional (ISO/IEC 19505) e é amplamente utilizada em projetos de pequena e grande escala.

Por Que a UML é Importante no Desenvolvimento de Sistemas?

- **Comunicação Clara entre Stakeholders** – Diagramas UML facilitam o entendimento entre desenvolvedores, analistas, clientes e outros envolvidos no projeto.
- **Documentação Eficiente** – Ajuda na manutenção e evolução futura do software.
- **Identificação de Requisitos e Erros Antecipadamente** – Modelar antes de codificar reduz retrabalho e custos.
- **Padronização Internacional** – Como linguagem universal, é reconhecida e aplicada em empresas e projetos no mundo todo.

Principais Diagramas UML e Suas Aplicações

- **Diagrama de Casos de Uso:** Define a interação entre usuários e sistema.
- **Diagrama de Classes:** Representa a estrutura estática do sistema (classes, atributos e métodos).
- **Diagrama de Sequência:** Mostra a ordem das interações entre objetos em um cenário específico.
- **Diagrama de Atividades:** Ilustra fluxos de processos e lógica de negócios.
- **Diagrama de Componentes:** Modela a estrutura física e a organização do código de um sistema de software
- **Diagrama de Implementação:** Mostra a arquitetura física de um sistema, ilustrando como os componentes de software são distribuídos sobre os recursos de hardware

O que é um diagrama de casos de uso em UML?

Na Linguagem de modelagem unificada (UML), o diagrama de caso de uso resume os detalhes dos usuários do seu sistema (também conhecidos como atores) e as interações deles com o sistema. Para criar um, use um conjunto de símbolos e conectores especializados. Um bom diagrama de caso de uso ajuda sua equipe a representar e discutir:

- Cenários em que o sistema ou aplicativo interage com pessoas, organizações ou sistemas externos
- Metas que o sistema ou aplicativo ajuda essas entidades (conhecidas como atores) a atingir
- O escopo do sistema

Quando usar o diagrama de caso de uso

O diagrama de caso de uso não oferece muitos detalhes não espere, por exemplo, que ele mostre a ordem em que os passos são executados. Em vez disso, um diagrama de caso de uso adequado dá uma visão geral do relacionamento entre casos de uso, atores e sistemas. Os especialistas recomendam usar o diagrama de caso de uso para complementar um caso de uso descrito em texto.

UML é o kit de ferramentas de modelagem para criar o diagrama. O caso de uso é representado por uma forma oval rotulada. Bonecos palito representam os atores no processo, e a participação do ator no sistema é modelada com uma linha entre o ator e o caso de uso. Para representar o limite do sistema, desenhe uma caixa em torno do próprio caso de uso.

O diagrama de caso de uso UML é ideal para:

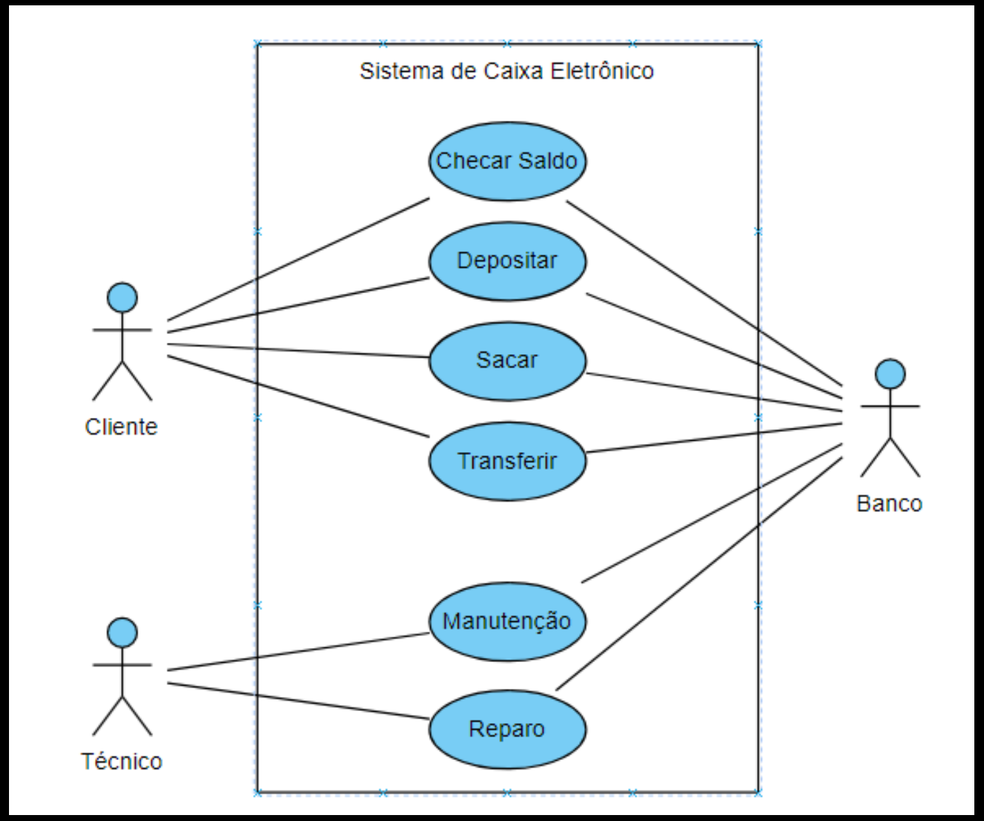
- Representar as metas de interações entre sistemas e usuários
- Definir e organizar requisitos funcionais no sistema
- Especificar o contexto e os requisitos do sistema
- Modelar o fluxo básico de eventos no caso de uso

Símbolos e notação no diagrama de caso de uso

- **Caso de uso:** formato oval na horizontal e que representam os diferentes usos que um usuário pode ter.
- **Atores:** bonecos palito, representando as pessoas que realmente implementam os casos de uso.
- **Associações:** uma linha entre atores e casos de uso. Nos diagramas complexos, é importante saber quais atores estão associados a quais casos de uso.
- **Caixa de limite do sistema:** caixa que define um escopo do sistema para os casos de uso. Todos os casos de uso fora da caixa são considerados fora do escopo do sistema. Por exemplo: o Psycho Killer está fora do escopo de ocupações no exemplo da serra elétrica abaixo.
- **Pacote:** uma forma UML na qual colocar diferentes elementos em grupos. Assim como no diagrama de componentes, esses agrupamentos são representados como pastas de arquivos. Assim como no diagrama de componentes, esses agrupamentos são representados como pastas de arquivos.

Exemplo: Caixa eletrônico

Nesta modelagem em caso de uso um Cliente de Banco faz o acesso a uma caixa eletrônico e pode fazer as seguintes operações financeiras: checar saldo, depositar, sacar e transferir. Além do Cliente e Banco, um outro ator é o Técnico que pode fazer manutenção ou reparo neste sistema. Elabore o caso de uso.



O que é um diagrama de classes UML?

Na taxonomia mais ampla da UML, os diagramas de classes se enquadram na família de diagramas estruturais (estáticos), que se concentram no que um sistema é feito, e não em como ele se comporta ao longo do tempo.

A UML foi estabelecida como um modelo padronizado para descrever uma abordagem de programação orientada a objetos. Como as classes são os blocos de construção dos objetos, os diagramas de classes são os blocos de construção da UML. Os vários componentes em um diagrama de classes podem representar as classes que serão programadas de fato, os objetos principais ou as interações entre classes e objetos. Como se alinham estreitamente com a programação orientada a objetos, os diagramas de classes geralmente podem ser traduzidos diretamente em construtos de código em linguagens OOP comuns, como Python, Java e C++.

A forma da classe em si consiste em um retângulo com três linhas. A linha superior contém o nome da classe, a linha central contém os atributos da classe e a seção inferior expressa os métodos ou operações que a classe pode usar. Classes e subclasses são agrupadas para mostrar o relacionamento estático entre cada objeto.

Benefícios dos diagramas de classes

Os diagramas de classes oferecem vários benefícios para qualquer organização. Use diagramas de classes UML para:

- Ajudar os desenvolvedores a pensar na estrutura e nas interações das classes antes de escrever qualquer código. Isso pode resultar em um software mais limpo e com melhor funcionamento.
- Desenhar gráficos detalhados e documentar requisitos que precisam ser incluídos no código.
- Compartilhar informações com as partes interessadas de forma mais fácil com apresentações visuais, sem precisar se aprofundar no código.
- Ilustrar modelos de dados para qualquer sistema, por mais complexo que seja.
- Compartilhar facilmente informações em toda a sua empresa.
- Ter uma melhor compreensão dos esquemas gerais de um aplicativo.
- Documentar requisitos e outros aspectos de um sistema sem explicações complicadas e cheias de texto.
- Criar modelos a partir de uma perspectiva de negócios como analista de negócios.

Componentes básicos de um diagrama de classes

Dependendo do contexto, as classes em um diagrama de classes podem representar os objetos principais, as interações no aplicativo ou as classes a serem programadas.

- Classes: um modelo para criar objetos e implementar comportamento em um sistema. Na UML, uma classe representa um objeto ou um conjunto de objetos que compartilham uma estrutura e um comportamento comuns. Elas são representadas por um retângulo que inclui linhas com o nome da classe, seus atributos e suas operações. Ao desenhar uma classe em um diagrama de classes, você só precisa preencher a linha superior as outras são opcionais caso queira fornecer mais detalhes.
- Sinais: símbolos que representam comunicações assíncronas e unidirecionais entre objetos ativos.
- Tipos de dados: classificadores que definem valores de dados. Os tipos de dados podem modelar tanto tipos primitivos quanto enumerações.
- Pacotes: formas projetadas para organizar classificadores relacionados em um diagrama. Eles são simbolizados por uma forma de retângulo grande com guias.

Seções de uma classe

Modificadores de acesso de membros

Todas as classes têm níveis de acesso diferentes dependendo do modificador de acesso (visibilidade). Aqui estão os níveis de acesso com seus respectivos símbolos:

- Público (+)
- Privado (-)
- Protegido (#)
- Pacote (~)
- Derivado (/)
- Estático (sublinhado)

Escopos de membros

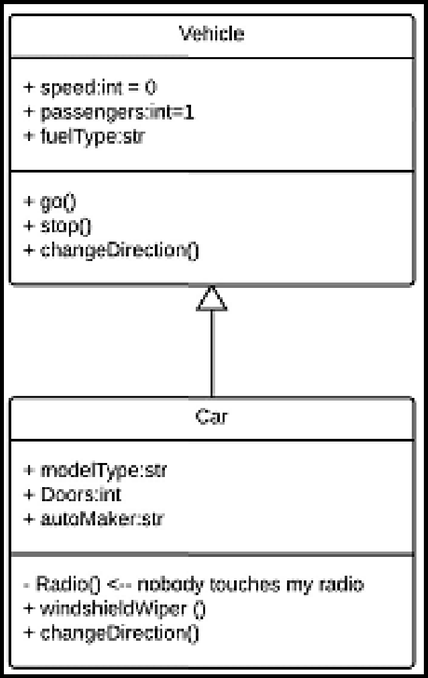
Existem dois escopos para membros: classificadores e instâncias. Os classificadores costumam ser reconhecidos como estáticos, o que significa que os valores dos atributos permanecem os mesmos em todas as instâncias e a invocação do método não afeta o estado da instância. Em comparação, as instâncias contêm métodos que podem alterar o estado da instância e valores de atributos que podem variar entre as instâncias. Para indicar um classificador, basta sublinhar seu nome; deixe o texto sem formatação para indicar uma instância.

Seções de uma classe

Interações

O termo "interações" refere-se aos vários relacionamentos e links que podem existir em diagramas de classes e objetos. Algumas das interações mais comuns incluem:

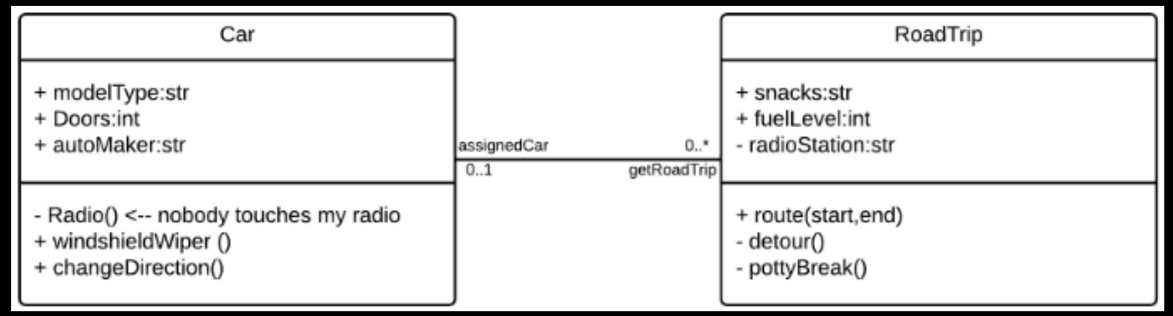
Herança: o processo de uma classe filha ou subclasse assumir a funcionalidade de uma classe pai ou superclasse, também conhecida como generalização. É simbolizada por uma linha reta conectada com uma ponta de seta fechada apontando para a superclasse.



Neste exemplo, o objeto "Carro" herdaria todos os atributos (velocidade, número de passageiros e combustível) e métodos (go(), stop(), changeDirection()) da classe pai ("Veículo"), além dos atributos específicos (tipo de modelo, número de portas e fabricante) e métodos de sua própria classe (Radio(), windshieldWiper(), ac/heat()). A herança é mostrada em um diagrama de classes usando uma linha sólida com uma seta fechada e vazia.

Seções de uma classe

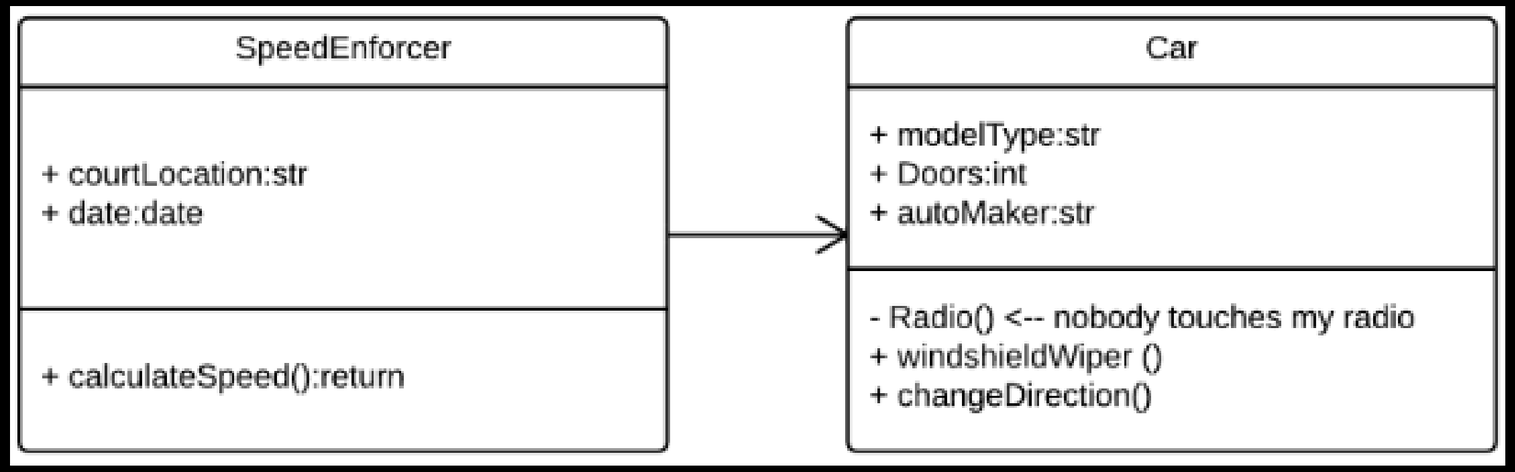
Associação bidirecional: o relacionamento padrão entre duas classes. Ambas as classes têm conhecimento uma da outra e de seu relacionamento com a outra. Essa associação é representada por uma linha reta entre duas classes.



No exemplo acima, a classe Car e a classe RoadTrip estão inter-relacionadas. Em uma extremidade da linha, o Car assume a associação de "assignedCar" com o valor de multiplicidade de 0..1, de modo que, quando a instância de RoadTrip existir, ela poderá ter uma instância de Car associada a ela ou nenhum Car associado a ela. Nesse caso, uma classe Caravan separada com um valor de multiplicidade de 0..* é necessária para demonstrar que uma RoadTrip poderia ter múltiplas instâncias de Cars associadas a ela. Como uma instância de Car poderia ter múltiplas associações "getRoadTrip" em outras palavras, um carro poderia fazer várias viagens o valor da multiplicidade é definido como 0..*

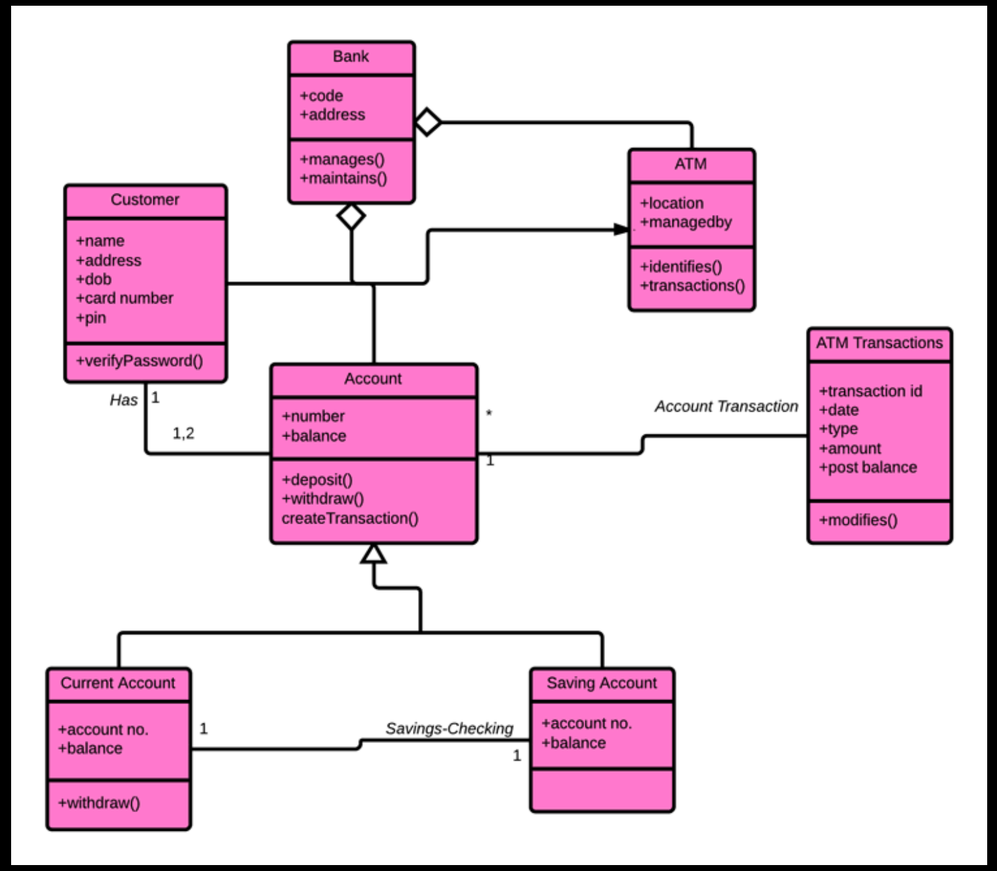
Seções de uma classe

Associação unidirecional: um relacionamento um pouco menos comum entre duas classes. Uma classe tem conhecimento da outra e interage com ela. A associação unidirecional é modelada com uma linha de conexão reta que aponta uma ponta de seta aberta da classe que conhece para a classe conhecida.



Como exemplo, em sua viagem pelo Arizona, você pode se deparar com um radar de velocidade onde uma câmera registra sua atividade de direção, mas você não saberá disso até receber uma notificação pelo correio. Não está desenhado na imagem, mas, neste caso, o valor da multiplicidade seria 0..*, dependendo de quantas vezes você passar pelo radar.

Diagrama de classes para um sistema de caixa eletrônico (ATM)



Os caixas eletrônicos (ATMs) são ilusoriamente simples: embora os clientes precisem apenas pressionar alguns botões para receber o dinheiro, existem muitas camadas de segurança pelas quais um caixa eletrônico seguro e eficaz deve passar para evitar fraudes e oferecer valor aos clientes bancários. As várias partes humanas e inanimadas de um sistema de caixa eletrônico são ilustradas por este diagrama de fácil leitura — cada classe tem seu título e os atributos estão listados abaixo. Você pode editar, salvar e compartilhar este gráfico abrindo o documento e se cadastrando em uma conta gratuita do Lucidchart.

O que é um diagrama de sequência em UML?

Diagrama de sequência é um tipo de diagrama de interação, pois descreve como, e em qual ordem, um grupo de objetos trabalha em conjunto. Estes diagramas são usados por desenvolvedores de software e profissionais de negócios para entender os requisitos de um novo sistema ou para documentar um processo existente. Diagramas de sequência são conhecidos como diagramas de eventos ou cenários de eventos.

Diagramas de sequência são úteis principalmente para visualizar interações ao longo do tempo em arquiteturas complexas, mantendo o fluxo compreensível tanto para partes interessadas técnicas quanto não técnicas.

Os benefícios de diagramas de sequência

Diagramas de sequência podem ser referências úteis para empresas e outras organizações. Experimente desenhar um diagrama de sequência para:

- Representar os detalhes de um caso de uso UML.
- Modelar a lógica de um processo, função ou operação sofisticado.
- Ver como objetos e componentes interagem uns com os outros para concluir um processo.
- Planejar e compreender a funcionalidade detalhada de um cenário existente ou futuro.

Casos de uso de diagramas de sequência

Os seguintes cenários são ideais para o uso de um diagrama de sequência:

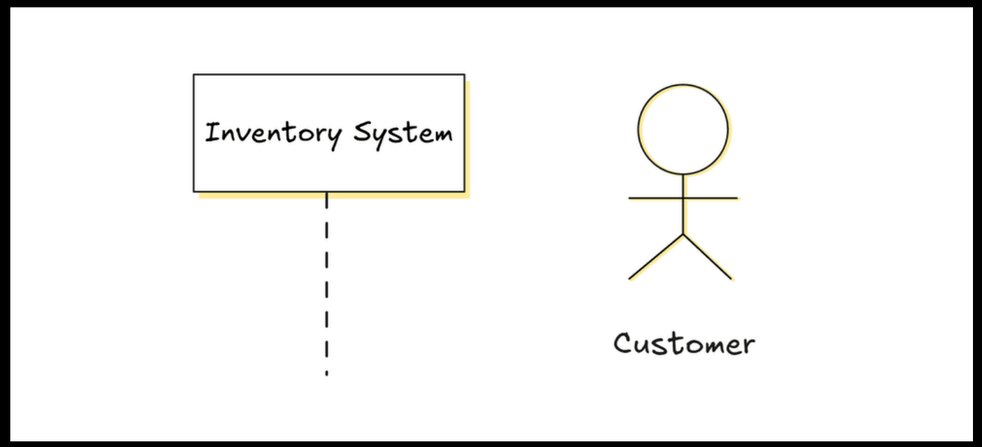
- Cenário de uso: um cenário de uso é um diagrama de como seu sistema poderia ser utilizado. É ótimo para certificar-se de que você compreendeu a lógica de cada cenário de uso do sistema.

- Lógica do método: assim como você usaria um diagrama de sequência UML para explorar a lógica de um caso de uso, você pode usá-lo para explorar a lógica de qualquer função, procedimento ou processo complexo.

- Lógica de serviço: se você acredita que serviços são métodos de alto nível usados por clientes, um diagrama de sequência é ideal para mapeá-lo.

- Diagrama de sequência do Visio - qualquer diagrama de sequência que você criar com o Visio também pode ser aberto no Lucidchart. O Lucidchart suporta a importação de arquivos .vsd e .vdx e é uma ótima alternativa ao Microsoft Visio. Quase todas as imagens que você vê na seção UML deste site foram geradas usando o Lucidchart.

Símbolos e componentes básicos



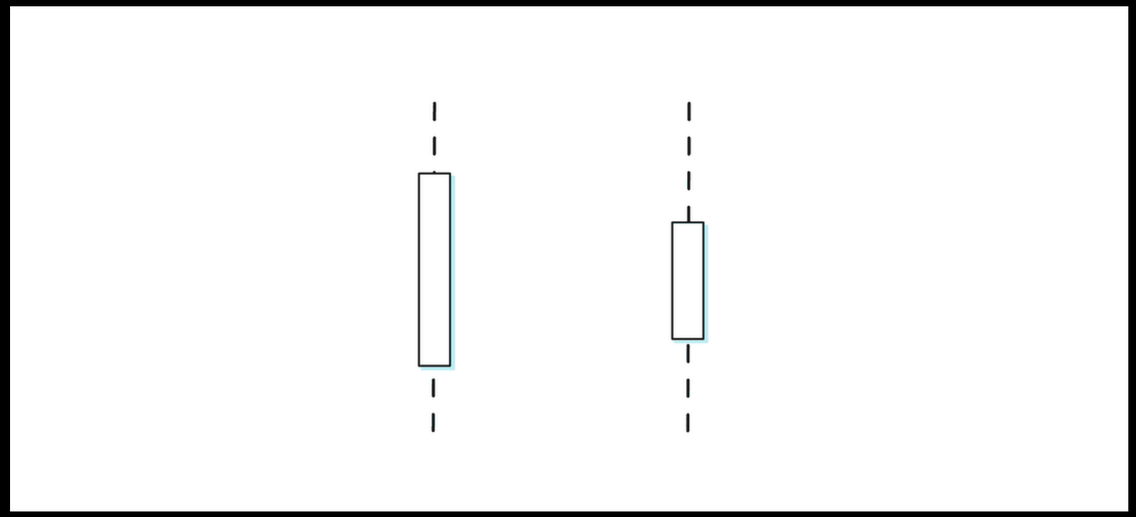
Linhas de vida e atores

Às vezes chamado de objeto, um indicador de desempenho (retângulo com uma linha descendente) é um elemento que participa de uma interação, como um carrinho de compras ou um sistema de pagamento.

Embora a linha geralmente seja tracejada, em algumas estruturas (como a Mermaid), ela pode ser contínua. Ambas são aceitáveis, desde que você seja consistente.

Um ator (figura de palito) é uma entidade externa que interage com o sistema. Normalmente, trata-se do usuário, por exemplo, um cliente que inicia o processo fazendo um pedido.

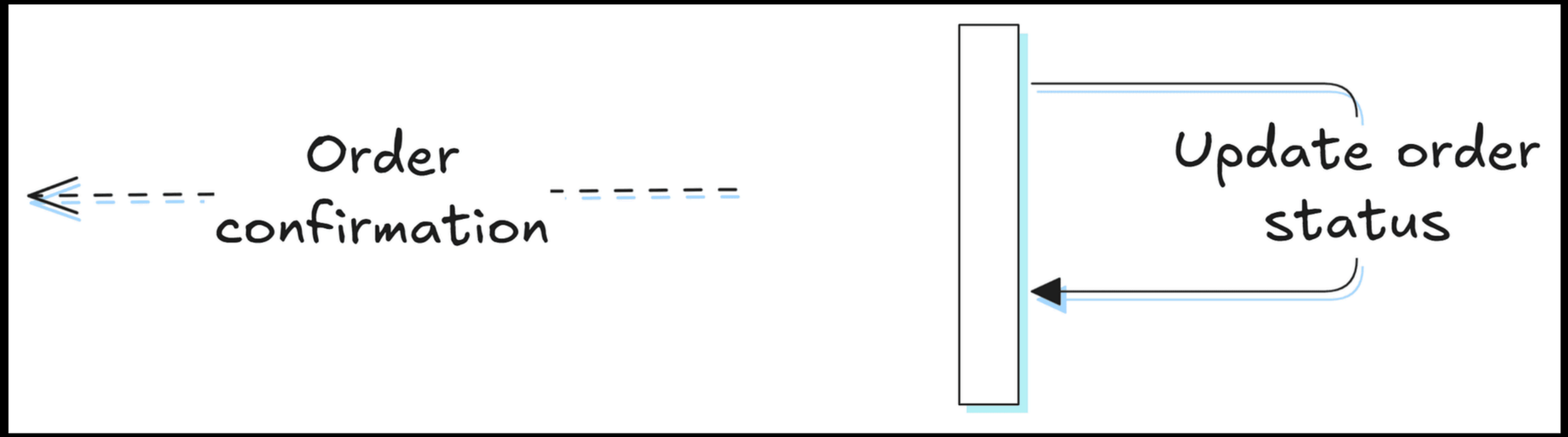
Símbolos e componentes básicos



Caixas de ativação

Também conhecido como ativação, o retângulo fino posicionado sobre as linhas verticais indica que o objeto está envolvido na operação iniciada por uma mensagem. Quanto mais tempo a tarefa levar, maior será o retângulo de ativação.

Símbolos e componentes básicos



Mensagens de retorno e auto-mensagens

Uma mensagem de retorno (seta tracejada com ponta tracejada) é enviada quando o receptor termina de processar uma mensagem e devolve o controle ao chamador.
Uma mensagem interna (uma seta em forma de U) é uma mensagem que um objeto envia para si mesmo. Geralmente representa uma operação interna.

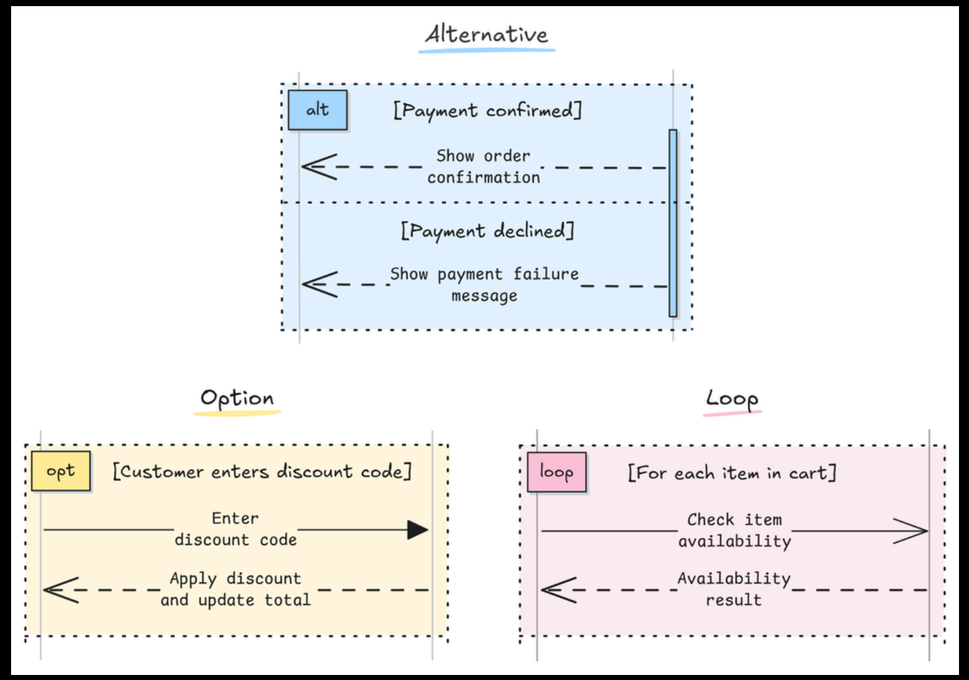
Símbolos e componentes básicos

Criar e destruir mensagens

Uma mensagem de criação adiciona um novo objeto ao diagrama de sequência. Ela é anexada à linha de vida que cria.

Uma mensagem de destruição faz o oposto, removendo um objeto existente. Ela é marcada com um X anexado à linha de vida destruída.

Símbolos e componentes básicos



Alternativas, opções e ciclos

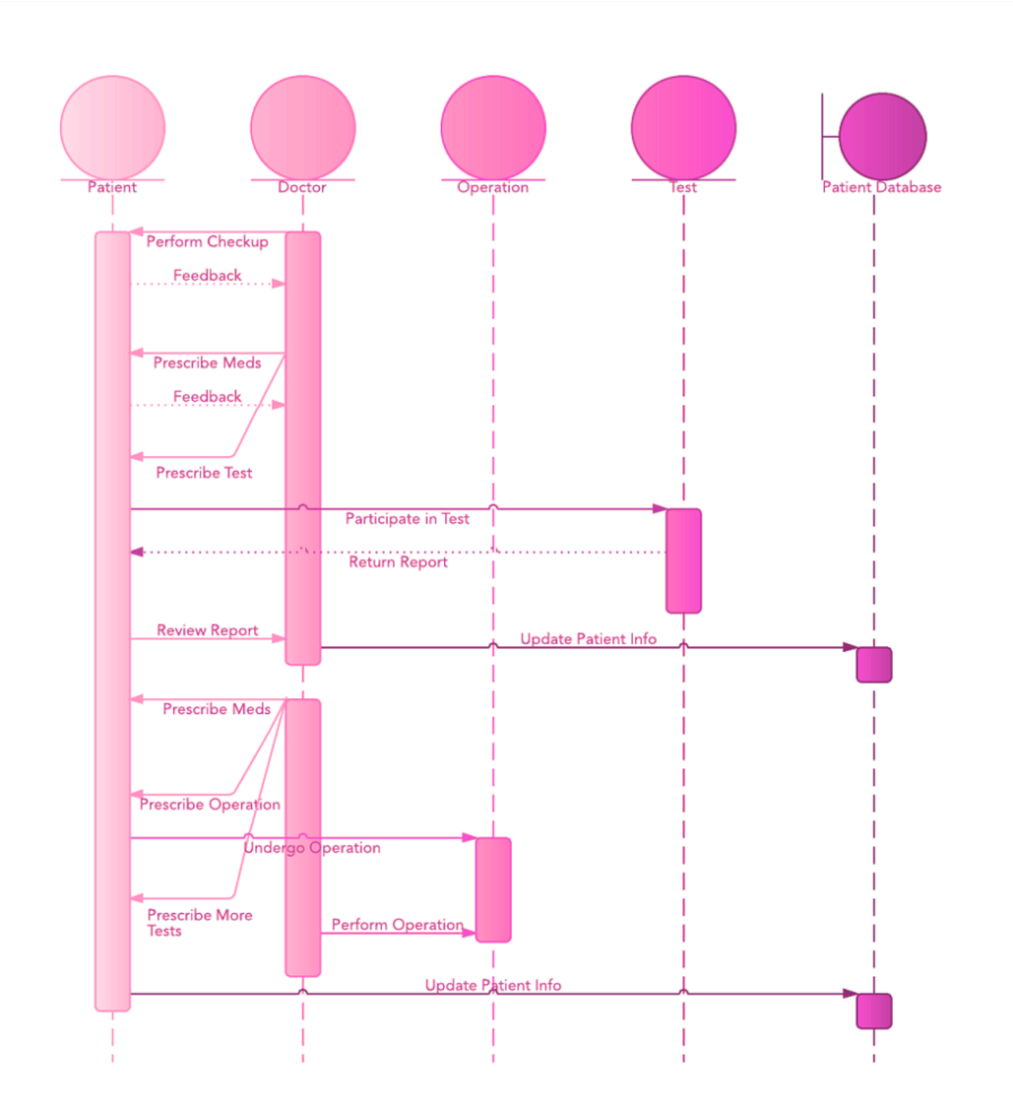
Um quadro alternativo (alt) mostra possíveis fluxos de interação, onde apenas um é executado se uma condição for atendida.

Uma opção (opt) representa um único caminho que é executado somente se sua condição for verdadeira.

Um loop é uma sequência de mensagens que se repetem até que uma condição seja atendida.

Os três elementos incluem texto entre colchetes para fornecer contexto adicional, como a condição necessária ou a ação que inicia o processo.

Diagrama de sequência de um sistema de gestão hospitalar



A tecnologia transformou completamente a medicina, assim como a maioria dos outros setores. Um sistema de gestão hospitalar, também conhecido como sistema de informações hospitalares, ajuda médicos, administradores e funcionários do hospital a gerenciar todas as atividades e informações obtidas em um hospital, incluindo exames, prescrições, consultas e informações sobre os pacientes e seus cuidadores. O diagrama abaixo é uma visualização simples de como os processos básicos funcionam uns com os outros ao longo do tempo.

O que é um diagrama de atividade?

A Linguagem de Modelagem Unificada inclui vários subconjuntos de diagramas, incluindo diagramas de estrutura, diagramas de interação e diagramas de comportamento. Os diagramas de atividades, juntamente com os diagramas de caso de uso e de máquina de estados, são considerados diagramas de comportamento porque descrevem o que deve acontecer no sistema que está sendo modelado.

As partes interessadas lidam com muitas questões, portanto, é importante se comunicar com clareza e concisão. Diagramas de atividade ajudam a unir as pessoas das áreas de negócios e de desenvolvimento de uma organização para entender o mesmo processo e comportamento. Para criar um diagrama de atividade, é necessário um conjunto de símbolos especiais, incluindo aqueles para dar partida, encerrar, fundir ou receber etapas no fluxo.

Benefícios de diagramas de atividade

Diagramas de atividade oferecem uma série de benefícios para os usuários. Considere criar um diagrama de atividade para:

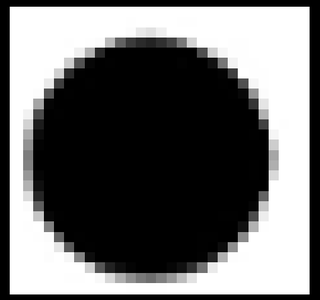
- Demonstrar a lógica de um algoritmo.
- Descrever as etapas realizadas em um caso de uso UML.
- Ilustrar um processo de negócio ou fluxo de trabalho entre usuários e o sistema.
- Simplificar e melhorar qualquer processo ao esclarecer casos de uso complicados.
- Modelar elementos de arquitetura de software, como método, função e operação.

Componentes básicos de um diagrama de atividade

Antes de criar um diagrama de atividade, é necessário primeiro entender sua composição. Alguns dos componentes mais comuns de um diagrama de atividade incluem:

- Ação: uma etapa na atividade em que os usuários ou o software realizam uma determinada tarefa.
- Nó de decisão: um ramo condicional no fluxo representado por um diamante. Inclui uma única entrada e duas ou mais saídas.
- Fluxos de controle: outro nome dado aos conectores que mostram o fluxo entre as etapas no diagrama.
- Nó inicial: simboliza o início da atividade. É representado por um círculo preto.
- Nó final: representa a etapa final da atividade. O nó final é representado por um círculo preto contornado.

Símbolos de diagramas de atividade



Símbolo de início

Representa o começo de um processo ou fluxo de trabalho em um diagrama de atividade. Ele pode ser usado por si só ou com um símbolo de nota que explica o ponto de partida.

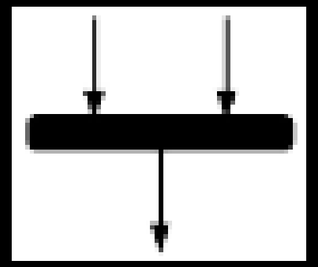
Símbolo de atividade

Indica as atividades que compõem um processo modelado. Esses símbolos, que incluem descrições curtas dentro da forma, são os principais componentes de um diagrama de atividades.

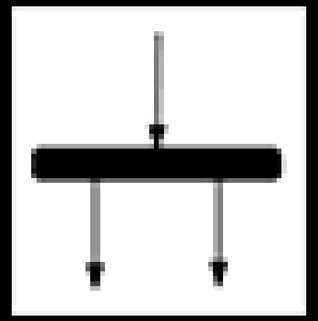
Símbolo de conector

Mostra o fluxo de direção, ou fluxo de controle, da atividade. Uma seta de entrada inicia um passo de uma atividade. Uma vez concluído o passo, o fluxo continua com a seta de saída.

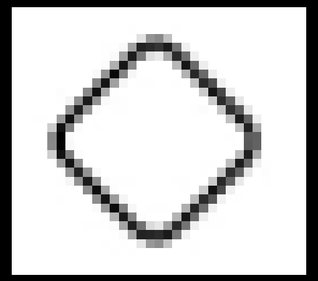
Símbolos de diagramas de atividade



Símbolo de junção ou barra de sincronização
Combina duas atividades simultâneas e as reintroduz em um fluxo onde apenas uma atividade ocorre por vez. Representado por uma linha espessa vertical ou horizontal.

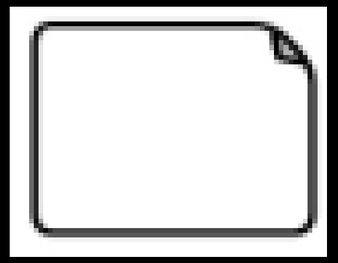


Símbolo de garfo
Divide um único fluxo de atividade em duas atividades simultâneas. É representado por várias linhas com setas partindo de uma junção.



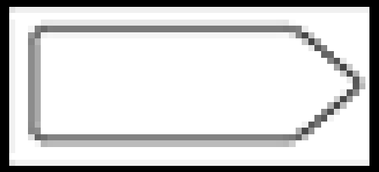
Símbolo de decisão
Representa uma decisão e sempre tem pelo menos dois caminhos ramificados e com texto de condição, permitindo aos usuários visualizarem opções. Este símbolo representa a ramificação ou fusão de diferentes fluxos, com o símbolo atuando como um quadro ou contêiner.

Símbolos de diagramas de atividade



Símbolo de nota

Permite aos criadores ou colaboradores do diagrama comunicar mensagens adicionais que não se encaixam dentro do próprio diagrama. Deixe observações para uma maior clareza e especificação.



Símbolo de enviar sinal

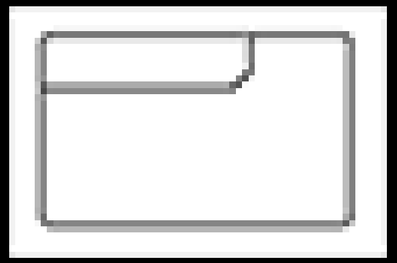
Indica que um sinal está sendo enviado a uma atividade recebedora.

Símbolo de receber sinal

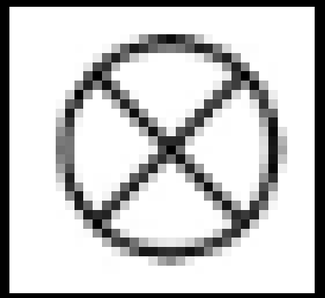
Demonstra a aceitação de um evento. Após o evento ser recebido, o fluxo que vem desta ação é concluído.

Símbolos de diagramas de atividade

Símbolo de história rasa pseudoestado
Representa uma transição que invoca o último estado ativo.



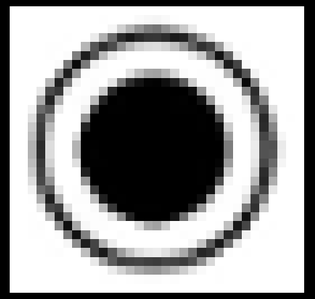
Símbolo de opção em loop
Permite ao criador modelar uma sequência repetitiva dentro do símbolo de opção em loop.



Símbolo de final de fluxo
Representa o final de um fluxo de processo específico. Este símbolo não deve representar o fim de todos os fluxos em uma atividade. Nesse caso, use o símbolo de término. O símbolo final do fluxo deve ser colocado no final de um processo em um fluxo único de atividade.

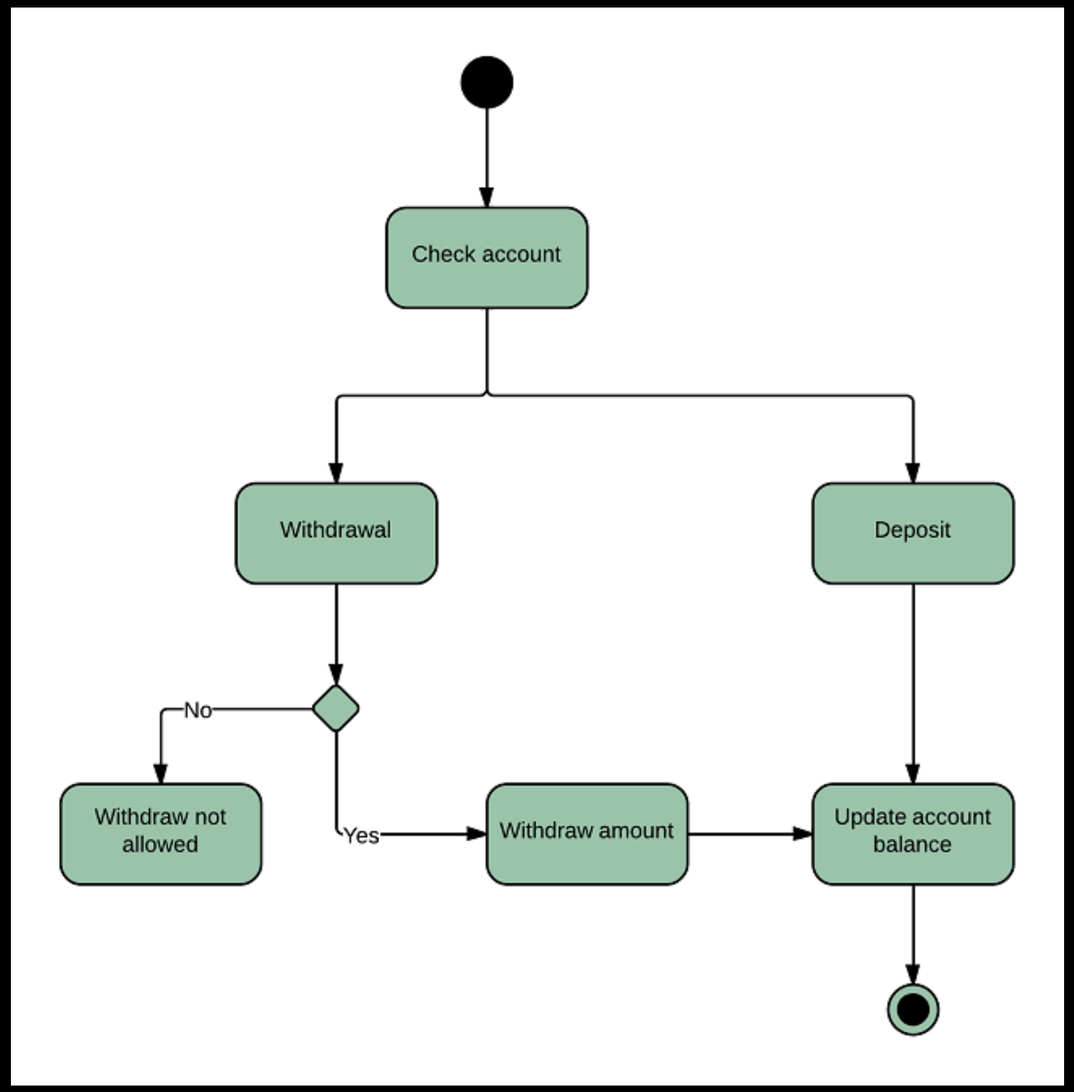
Símbolos de diagramas de atividade

Texto de condição
É colocado ao lado de um marcador de decisão para avisar em qual condição um fluxo de atividade deve se separar nesse sentido.



Símbolo de término
Marca o estado final de uma atividade e representa a conclusão de todos os fluxos de um processo.

Diagrama de atividade para um sistema bancário



Este diagrama mostra o processo de saque ou depósito de dinheiro em uma conta bancária. A vantagem de representar o fluxo de trabalho visualmente em UML é a possibilidade de exibir saques e depósitos em um único gráfico. Ao usar o Lucidchart para criar um diagrama de atividade, você pode personalizar seus modelos com fontes e cores profissionais. Com o armazenamento seguro e baseado em nuvem, você nunca precisará se preocupar em perder seus documentos.

O que é diagrama de componentes UML?

O diagrama de componentes mostra o relacionamento entre diferentes componentes de um sistema. Para fins de UML 2.0, o termo "componente" refere-se a um módulo de classes que representa sistemas ou subsistemas independentes com capacidade de interagir com o restante do sistema.

Para isso, existe uma abordagem de desenvolvimento em torno de componentes: o desenvolvimento baseado em componentes (CBD). Nela, o diagrama de componentes identifica os diferentes componentes para que todo o sistema funcione corretamente.

O mais comum, em uma abordagem de programação OO, é o desenvolvedor sênior usar o diagrama de componentes para agrupar classes com base em um objetivo comum, para que o desenvolvedor e outras pessoas envolvidas analisem o projeto de desenvolvimento de software de forma generalizada.

Benefícios do diagrama de componentes

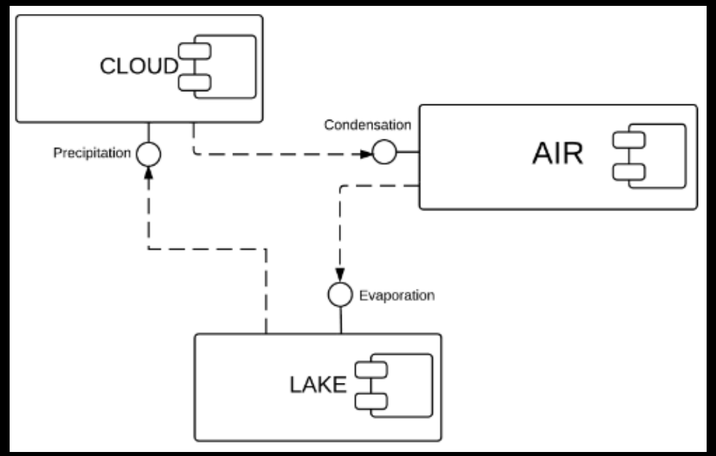
Embora o diagrama de componentes pareça complexo à primeira vista, ele é indispensáveis na criação do seu sistema. Com ele, sua equipe:

- Imagina a estrutura física do sistema.
- Presta atenção aos componentes do sistema e como eles se relacionam.
- Enfatiza o comportamento do serviço quanto à interface.

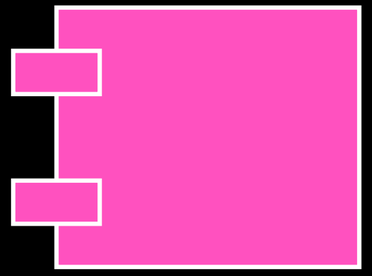
Como usar o diagrama de componentes diagrama de componentes

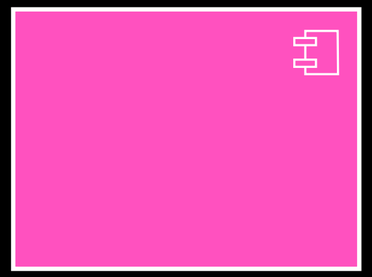
O diagrama de componentes em UML dá uma visão geral do sistema de software. Compreender o comportamento exato do serviço fornecido por cada software fará de você um desenvolvedor melhor. O diagrama de componentes descreve sistemas de software implementados em qualquer linguagem ou estilo de programação.

A Linguagem de modelagem unificada (UML) é um conjunto de convenções para diagramas baseados em objetos e possui várias aplicações. No diagrama de componentes, a UML determina que componentes e pacotes sejam conectados a linhas que representam conectores de montagem e de delegação.



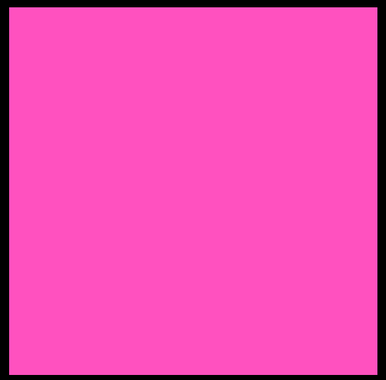
Formas e símbolos do diagrama de componentes





Símbolo de componente

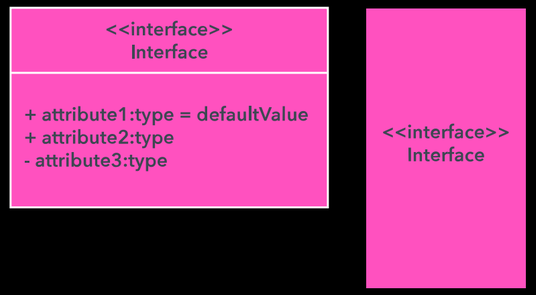
Entidade necessária para executar uma função de estereótipo. O componente fornece e consome comportamento em interfaces ou por meio de outros componentes. Considere os componentes como um tipo de classe. Na UML 1.0, o componente tem forma de bloco retangular, com dois retângulos menores saindo pela lateral. Na UML 2.0, o componente tem forma de bloco retangular com uma pequena imagem da forma antiga do diagrama de componentes.



Símbolo de nó

Representa objetos de hardware ou software, de nível superior aos componentes.

Formas e símbolos do diagrama de componentes

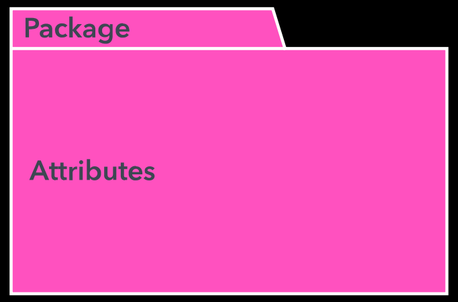


Símbolo de interface

Mostra entradas ou materiais que o componente recebe ou fornece. A interface pode ser representada por notas ou símbolos de texto, como formato de pirulito, soquete e "bola-e-soquete".

Símbolo de porta

Especifica um ponto de interação separado entre o componente e o ambiente. O símbolo da porta é um pequeno quadrado.



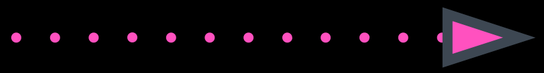
Símbolo de pacote

Agrupa vários elementos do sistema e é representado por pastas de arquivos. Assim como as pastas de arquivos agrupam várias folhas, o pacote pode ser desenhado em torno de vários componentes.

Formas e símbolos do diagrama de componentes

Símbolo de nota

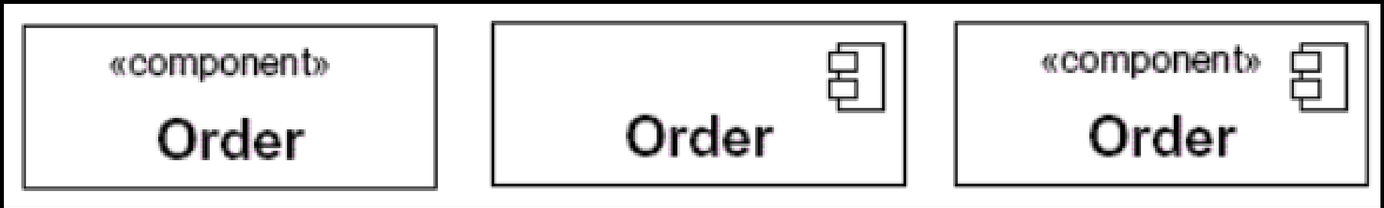
Com ele, os desenvolvedores fixam uma meta-análise no diagrama de componentes.



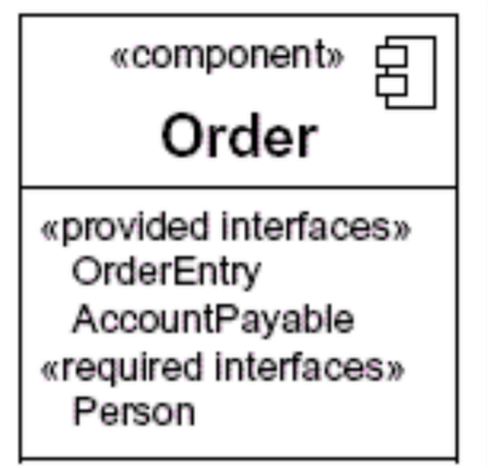
Símbolo de dependência

Mostra que uma parte do sistema depende de outra. A dependência é representada por linhas tracejadas que vinculam um componente (ou elemento) a outro.

Como usar formas e símbolos de componentes

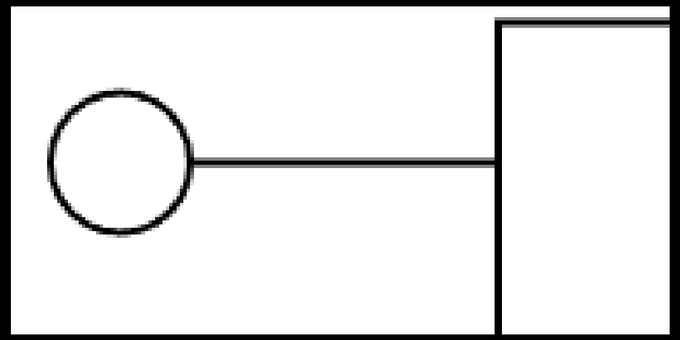


Existem três maneiras comuns de criar o compartimento de nome do componente. É sempre necessário incluir o texto do componente entre aspas angulares e/ou o logotipo do componente. Essa diferença é importante, pois o retângulo com apenas um nome dentro representa classificadores (elementos de classe).

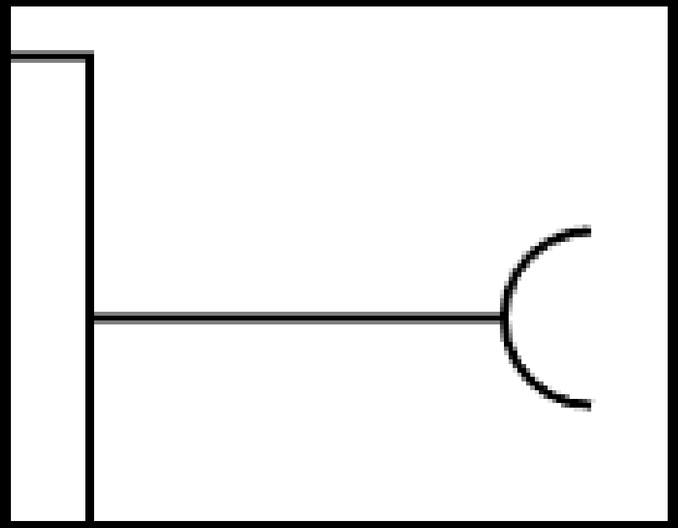


Assim como na notação de classe, o componente também tem um espaço para listar interfaces, semelhante a como atributos e métodos à notação de classe são adicionados. A interface representa os locais onde os grupos de classes no componente se comunicam com outros componentes do sistema. Outra maneira de representar a interface é estender um símbolo da caixa de componentes. Veja um rápido resumo dos símbolos mais usados.

Como usar formas e símbolos de componentes

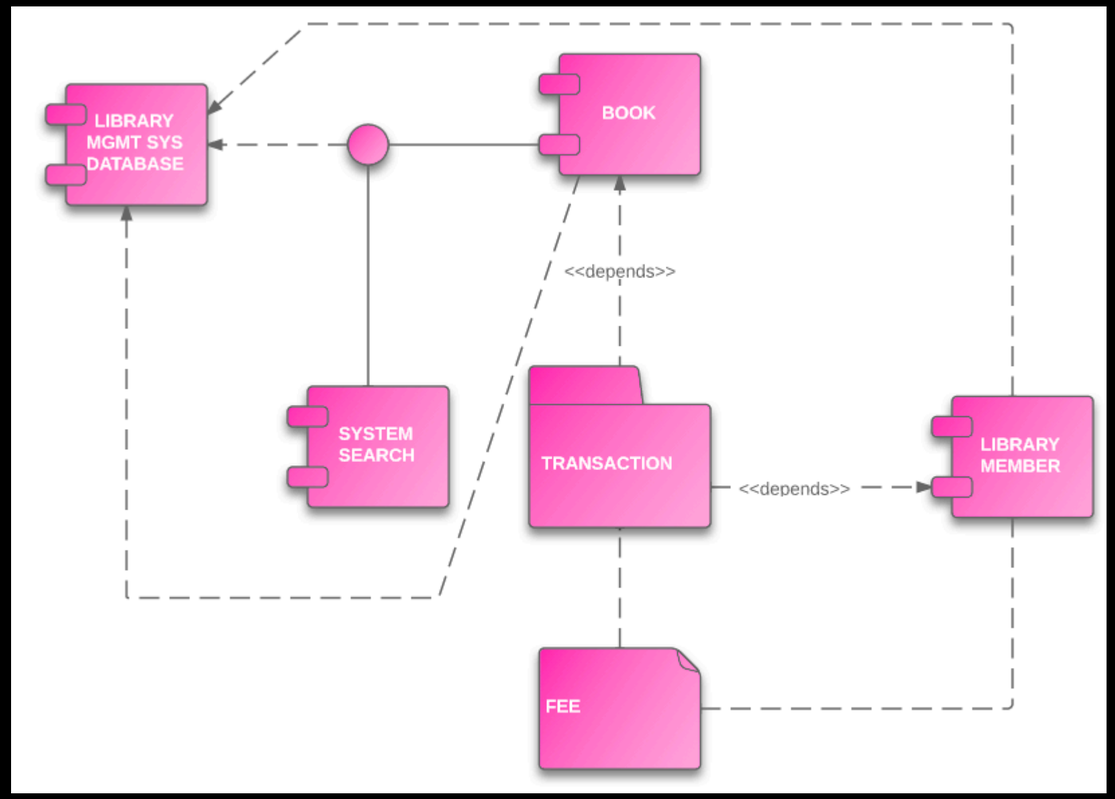


Interface fornecida: uma linha reta a partir da caixa de componentes e com um círculo anexado. Esse símbolo representa a interface na qual um componente produz informações usadas pela interface necessária de outro componente.



Interface necessária: uma linha reta da caixa de componentes e com um semicírculo anexado (também representado como uma seta tracejada com uma seta aberta). Esse símbolo representa a interface na qual um componente requer informações para executar a função correta.

Diagrama de componentes para um sistema de gerenciamento de bibliotecas



O sistema de bibliotecas foi um dos primeiros sistemas no mundo a ser amplamente gerenciado por computador.

Hoje em dia, muitos desses sistemas são gerenciados na nuvem por serviços terceirizados, não internamente. Embora o termo “sistema de biblioteca” normalmente remeta a uma maneira de monitorar livros impressos, os sistemas de bibliotecas modernos organizam todos os dados registrados por usuários.

Essas transações criam uma rede de relacionamentos entre os componentes do sistema da biblioteca. Veja no diagrama UML abaixo como os relacionamentos e o sistema funcionam. Você ou sua equipe também pode editar este modelo pronto.

O que é um diagrama de implementação?

No contexto da Linguagem de modelagem unificada (UML), um diagrama de implementação faz parte da família de diagramação estrutural pois descreve um aspecto do sistema em si. Neste caso, o diagrama de implementação descreve a implementação física de informações geradas pelo programa de software em componentes de hardware. A informação gerada pelo software é chamada de artefato. Não se deve confundir este com o uso do mesmo termo em outras abordagens de modelagem, como BPMN.

Os diagramas de implementação são feitos de várias formas de UML. As caixas tridimensionais conhecidas como nós representam os elementos básicos de software ou hardware, ou nós do sistema. As linhas de nó a nó indicam relacionamentos e as formas menores contidas dentro das caixas representam os artefatos de software empregados. Por descreverem hardware, os diagramas de implantação são únicos no universo UML.

Aplicações do diagrama de implementação

Os diagramas de implementação possuem várias aplicações de grande valor. Eles podem ser usados para:

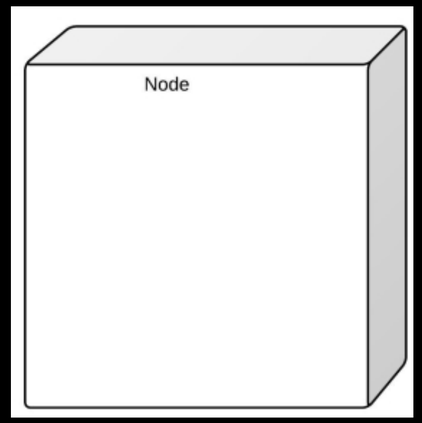
- Mostrar quais elementos de software são empregados por quais elementos de hardware.
- Ilustrar o processamento do tempo de execução do hardware.
- Propiciar uma visão da topologia do sistema do hardware.

Elementos do diagrama de implementação

Diagramas de implementação são constituídos por uma variedade de formas. Esta lista oferece uma visão geral dos elementos básicos que podem ser encontrados.

- Associação: uma linha que indica uma mensagem ou outro tipo de comunicação entre nós.
- Componente: um retângulo com duas abas que indica um elemento de software.
- Dependência: uma linha tracejada finalizada em uma seta, indicando que um nó ou componente é dependente de um outro.
- Interface: um círculo que indica uma relação contratual.São aqueles objetos que notam que a interface deve realizar algum tipo de obrigação.
- Nó: um objeto de hardware ou software representado por uma caixa tridimensional.
- Nó como contêiner: um nó que contém outro nó em seu interior, tal como no exemplo abaixo onde os nós contêm componentes.
- Estereótipo: um dispositivo contido dentro do nó, apresentado na parte superior do nó, com o nome entre setas duplas.

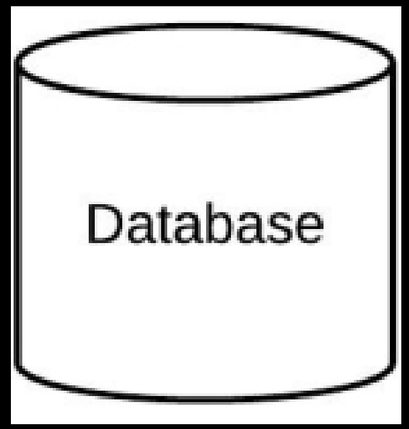
Símbolos e notação do diagrama de implementação



Nódulos (Nós)

Existem dois tipos de nós em um diagrama de implementação: nós de dispositivos e nós de ambiente de execução. Nós de dispositivos são recursos de computação com capacidades de processamento e execução de programas. Alguns exemplos de nós de dispositivos são PCs, laptops ou celulares.

Um nó de ambiente de execução, ou EEN, é qualquer sistema de computação que resida dentro de um nó de dispositivo. Pode ser um sistema operacional, um JVM ou outro recipiente web.



Base de dados

Bancos de dados representam qualquer dado armazenado pelo sistema implementado. Em alguns casos, o banco de dados é representado como um outro nó, no entanto, às vezes você verá esta forma representando um banco de dados.

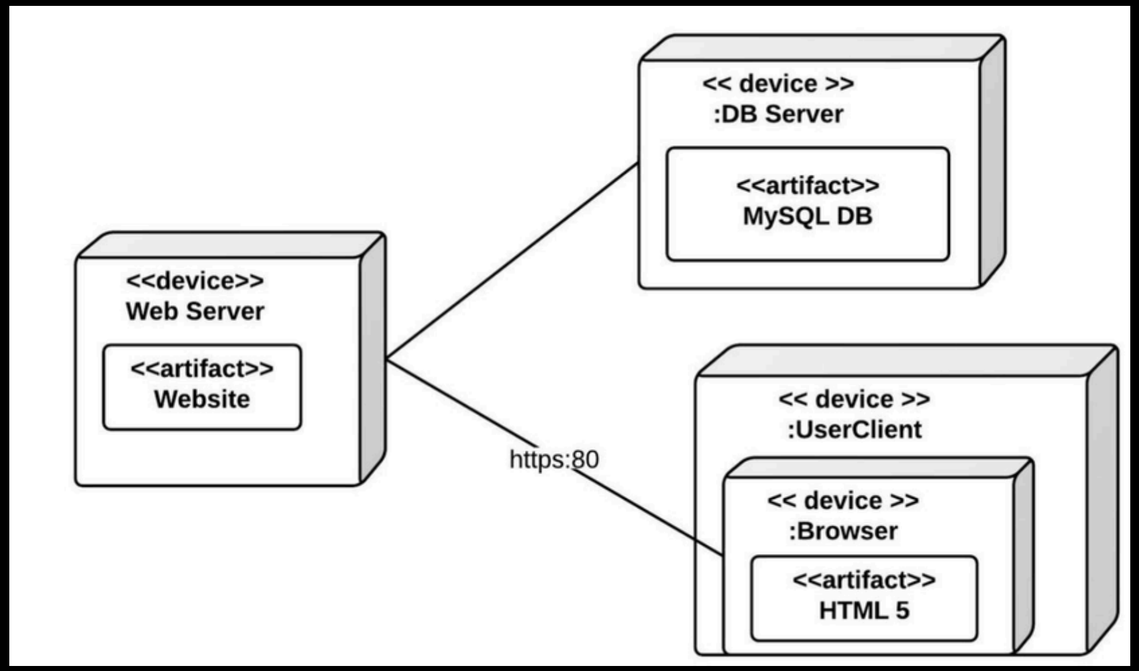
Símbolos e notação do diagrama de implementação

Outras formas:

- Percurso da comunicação: uma linha reta que representa a comunicação entre dois nódulos de aparelho.
- Artefatos: Uma caixa com o cabeçalho "<>" e depois o nome do arquivo.
- Pacote: uma caixa em forma de arquivo que agrupa todos os nós de dispositivos para integrar toda a implementação.
- Componente: uma entidade necessária para executar uma função de estereótipo.

Posicionando os símbolos, lembre-se de que diagramas de implementação podem variar do simples ao muito detalhado. Por exemplo, você pode mostrar um nó aninhado dentro de outro para refletir um padrão de implantação específico.

Exemplo de diagrama de implementação



Este exemplo mostra um diagrama básico de implementação do Lucidchart. Há um servidor web, um servidor de banco de dados e o aparelho de onde o usuário visualiza o site. É possível acrescentar uma maior complexidade mostrando as diferentes partes do servidor web e como o Javascript funciona no UserClient, mas este exemplo mostra um pouco como é a implementação ao usarmos a notação UML.

Questão 4

Durante o levantamento de requisitos de um complexo sistema de e-commerce, o Analista de Requisitos modelou os seguintes cenários no Diagrama de Casos de Uso:

- Ao realizar a ação "Finalizar Compra", o sistema invariavelmente executa a ação "Calcular Frete".
- Durante a ação "Finalizar Compra", o sistema pode, opcionalmente, executar a ação "Aplicar Cupom de Desconto", dependendo de uma condição específica inserida pelo usuário.
- O sistema possui dois perfis: "Cliente Ouro" e "Cliente Prata", ambos derivam de um perfil genérico chamado "Cliente".

Considerando as diretrizes da UML, analise as proposições sobre a modelagem desse cenário:

I. O relacionamento entre "Finalizar Compra" e "Calcular Frete" deve ser modelado com o estereótipo <<include>> (inclusão), pois indica um comportamento obrigatório e incorporado pelo caso de uso base.
II. O relacionamento entre "Finalizar Compra" e "Aplicar Cupom de Desconto" deve ser modelado com o estereótipo <<extend>> (extensão), pois o caso de uso estendido é acionado apenas se os pontos de extensão (extension points) e suas condições forem satisfeitos.
III. Os perfis "Cliente Ouro" e "Cliente Prata" devem estar ligados ao ator genérico "Cliente" através de um relacionamento de associação bidirecional, indicando troca de dados entre os atores.
IV. Atores em diagramas de casos de uso representam estritamente seres humanos; caso o sistema precise validar o pagamento com um "Gateway do Banco" externo, esse gateway deve ser modelado como uma classe interna e não como um ator.

É correto o que se afirma em:

A) I e II, apenas. B) II e III, apenas. C) I, II e IV, apenas. D) I, III e IV, apenas. E) I, II, III e IV.

Questão 4

Gabarito: A (I e II, apenas)

Justificativa:

- I está CORRETA. O <<include>> dita obrigatoriedade e reuso. O "Finalizar Compra" sempre invocará o "Calcular Frete".
- II está CORRETA. O <<extend>> é acionado de forma condicional/opcional. A aplicação de cupom depende de pontos de extensão.
- III está INCORRETA. Para mostrar que "Cliente Ouro" deriva de "Cliente", utiliza-se um relacionamento de Generalização/Herança (seta vazada), e não associação bidirecional.
- IV está INCORRETA. Na UML, "Atores" não são apenas humanos; sistemas corporativos externos, hardwares, serviços e APIs de terceiros (como o Gateway do banco) DEVEM ser modelados como atores se eles interagirem com as fronteiras do sistema.

Referências

- ALURA. O que é Metodologia Ágil. [S. l.], [s.d.]. Disponível em: https://www.alura.com.br/artigos/o-que-e-metodologia-agil?srsltid=AfmBOooL_9580NS2uE7446SzknyMKL2KuOULKkDzE-OXlfZdAGBqVGpO. Acesso em: 9 set. 2026.
- PERRELLA, Fabricio. Conceitos e Evolução da Engenharia de Software. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.
- PERRELLA, Fabricio. Modelagem de Software. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.
- PERRELLA, Fabricio. Princípios de Engenharia de Requisitos. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.
- PERRELLA, Fabricio. Processos de Software. São Caetano do Sul: Universidade Municipal de São Caetano do Sul (USCS), 2024. Slides de aula.
- PUCRS. Princípios do PMBOK. Blog PUCRS Online, [s.d.]. Disponível em: https://online.pucrs.br/blog/principios-do-pmbok. Acesso em: 9 set. 2026.
- ANTHROPIC. Claude. [Modelo de linguagem]. São Francisco: Anthropic, 2026. Disponível em: https://claude.ai. Acesso em: 9 set. 2026.
- GOOGLE. Gemini. [Modelo de linguagem]. Mountain View: Google, 2026. Disponível em: https://gemini.google.com. Acesso em: 9 set. 2026.
- Understanding the ISO/IEC 25010: framework for software quality https://www.sonarsource.com/resources/library/iso-iec-25010-explained/
- Introdução à Linguagem de Modelagem Unificada® (UML®) https://www.omg.org/uml/
- Tutorial sobre diagrama de caso de uso https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-caso-de-uso
- Tutorial de diagrama de classes https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-classes
- Tutorial de diagrama de sequência https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-sequencias
- Diagrama de sequência https://plus.excalidraw.com/use-cases/sequence-diagram#activation-boxes
- Tutorial sobre diagramas de atividades UML https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-atividades
- Diagrama de componentes UML: o que é, como fazer e exemplos https://lucid.co/pt/diagrama/uml/tutorial-de-diagrama-de-componentes

Os
originais

Obrigado!